

A PROJECT REPORT ON
SURVEILLANCE SPY ROBOT USING ESP32 WITH IOT-
BASED CONTROL

A Report Submitted to Jawaharlal Nehru Technological University, Kakinada in Partial
fulfilment of Requirements for the Award of the Degree of

BACHELOR OF TECHNOLOGY
IN
ELECTRONICS AND COMMUNICATION ENGINEERING
(2024-2025)



SUBMITTED BY

21HU1A0437	SHAIK BASHEER
22HU5A0405	GONISIPUDI SANKAR
21HU1A0422	KORUMALLI TEJA
21HU1A0448	VALLURU NARENDRA

Under The Esteemed Guidance Of

Mr. ANKA SIVA PRASAD KATURU MTech

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
CHEBROLU ENGINEERING COLLEGE

Accredited by NAAC 'A' GRADE

(Approved by AICTE New Delhi, Permanently Affiliated to JNTUK, Kakinada,
Post & Mandal, near Power Station, Chebrolu, Andhra Pradesh 522212)

CHEBROLU ENGINEERING COLLEGE
DEPARTMENT OF
ELECTRONICS AND COMMUNICATION ENGINEERING



CERTIFICATE

This is to certify that the project work entitled “**SURVEILLANCE SPY ROBOT USING ESP32 WITH IOT BASED CONTROL**” is a Bonafide record of work done by SHAIK BASHEER (21HU1A0437), GONISIPUDI SANKAR (22HU5A0405), KORUMALLI TEJA (21HUIA0422), VALLURU NARENDRA (21HUIA0448), Fulfilment for the award of the degree of Bachelor of Technology in ELECTRONICS AND COMMUNICATION ENGINEERING of Jawaharlal Nehru Technological University, Kakinada during the year 2021-2025. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the above degree.

PROJECT GUIDE

HEAD OF THE DEPARTMENT

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We Owe a great many Thanks to many people who helped and supported and suggested us in every step. We are glad for having the support of principal of our institution **Dr. R.V. Krishnaiah**, sir who inspired us with his words filled with dedication and discipline towards work.

We express our gratitude towards head of the department of electronics and communication engineering **Dr. S Satheesh** sir for extending his support through valuable suggestions which had been the major source to carry out our project.

We are very much thankful to **Mr. Anka Siva Prasad Katuru** Assistant Professor, Guide of our project for guiding and correcting various documents of ours with attention and care. He has taken pain to go through the project and make necessary corrections as and when needed. Finally, we thank one and all who directly and indirectly helped us to complete our project successfully.

Project Members:

21HU1A0437	SHAIK BASHEER
22HU5A0405	GONISIPUDI SANKAR
21HU1A0422	KORUMALLI TEJA
21HU1A0448	VALLURU NARENDRA

DECLARATION

We here by declare that the work which is being presented in this entitled **“SURVEILLANCE SPY ROBOT USING ESP32 WITH IOT BASED CONTROL”** Submitted towards the partial fulfilment of requirements for the award of the degree in record of our work carried out under the supervision of **Mr. Anka Siva Prasad Katuru** Assistant Professor in Department of Electronics and communication engineering. The matter embodied in this dissertation report has not been submitted by us. For the award of any degree. Further the technical details furnished in the various chapters in this report are purely relevant to the above project and there is no deviation from the theoretical point of view for design, development and implementation.

Project Members:

21HU1A0437 SHAIK BASHEER

22HU5A0405 GONISIPUDI SANKAR

21HU1A0422 KORUMALLI TEJA

21HU1A0448 VALLURU NARENDRA

INDEX

CHAPTER 1

INTRODUCTION.....1-4

1.1 Introduction.....	1
1.2 Problem Statement.....	1
1.3 Problem Solution.....	2
1.4 Key features off the Solution.....	2-3
1.5 Literature Survey.....	3-4

CHAPTER 2

BLOCK DIAGRAM & CIRCUIT DIAGRAM.....5

2.1 Block Diagram.....	5
2.2 Circuit Diagram.....	5

CHAPTER 3

COMPONENTS & ITS SPECIFICATIONS.....6-9

3.1 ESP32 Microcontroller.....	6-7
3.2 ESP32 CAM.....	7
3.3 LM2596 Power Supply Module.....	8-9
3.4 DC-DC Motor.....	9
3.5 L298N Motor Driver.....	9

CHAPTER 4

SOFTWARE & INSTALLATION PROCEDURE.....10-35

4.1 About Arduino IDE and its features.....	10-21
4.2 Arduino IDE.....	21-22
4.3 Why Arduino IDE for ESP32/ESP8266.....	22-30

CHAPTER 5

IOT CLOUD & SERVER PLATFORM.....31-51

CHAPTER 6

PROPOSED SOLUTION.....52

6.1 Working Principle of Ideation.....52

CHAPTER 7

OUTPUT & RESULTS.....53

7.1 Project Photos.....53

CHAPTER 8

CONCLUSION.....54

8.1 Project Conclusion.....54

CHAPTER 9

FUTURE SCOPE.....54

9.1 Future addons or Updates in project.....54

CHAPTER 10

SOURCE CODE & RESOURCES.....56-60

10.1 Code.....56-59

10.2 Resources.....59-60

CHAPTER 11

REFERENCES.....61

LIST OF FIGURES

Figure 2.1: Block Diagram	05
Figure 2.2: Circuit Diagram of ESP32.....	05
Figure 3.1: ESP32 Microcontroller.....	06
Figure 3.2: ESP32 CAM.....	07
Figure 3.3: LM Power Supply Module	08
Figure 3.4: L298N Motor Driver	09
Figure 4.1: Arduino IDE Screen	10
Figure 5.1: Publish-Subscribe Model.....	36
Figure 5.2: Template Creation of Blynk IoT.....	40
Figure 5.3: Installation of Blynk IoT in Mobile.....	46
Figure 7.1: Project Photos.....	53
Figure 7.2: Project Result.....	53

ABSTARCT

In today's world, surveillance and security are crucial in various sectors, including defence, industrial monitoring, and home security. This project focuses on designing and implementing a Surveillance Spy Robot using an ESP32, an ESP32-CAM, a motor driver, and an LM2596 power supply module. The robot is designed for remote monitoring and movement control, making it suitable for reconnaissance in hazardous environments. The ESP32 microcontroller serves as the core processing unit, enabling communication between the motor driver and the IoT-enabled Android application. The ESP32-CAM module provides live video streaming, allowing users to monitor real-time visuals remotely. A motor driver circuit is used to control the movement of the robot, and an LM2596 power module ensures a stable power supply. The robot is controlled using a Wi-Fi based Android application, allowing users to navigate the bot wirelessly. The ESP32 receives commands via Bluetooth, processes them, and controls the motors accordingly. The ESP32-CAM captures and transmits real-time video feed, enhancing surveillance capabilities. This project is ideal for security applications, military reconnaissance, and remote monitoring in inaccessible areas. It provides an efficient and cost-effective surveillance system with wireless control and live video streaming features.

CHAPTER-1

INTRODUCTION

1.1 Introduction:

In the modern era, surveillance and security have become essential aspects of various domains, including defense, industrial monitoring, and home security. Traditional surveillance systems often require extensive infrastructure, fixed installations, and high maintenance costs. To overcome these challenges, autonomous and remotely controlled surveillance systems are being developed. This project focuses on designing and implementing a Surveillance Spy Robot using an ESP32 microcontroller, an ESP32-CAM module, a motor driver, and an LM2596 power supply module. The robot is intended to provide real-time monitoring capabilities in hazardous or inaccessible environments. With its ability to stream live video and be remotely controlled via a Wi-Fi-based Android application, this system offers an efficient and cost-effective alternative to conventional surveillance mechanisms. By integrating wireless control and live video streaming, this project enhances security applications, military reconnaissance, and industrial monitoring solutions.

1.2 Problem Statement:

Security and surveillance systems are crucial in various applications, including military, industrial, and domestic environments. Conventional surveillance methods, such as fixed CCTV cameras, are often limited in terms of mobility, flexibility, and adaptability. These systems require significant infrastructural investment, and their fixed positioning prevents real-time tracking of suspicious activities in dynamic environments.

Moreover, certain areas such as hazardous industrial zones, military battlefields, and remote locations require mobile surveillance solutions that can navigate autonomously or be controlled remotely. There is a growing need for a cost-effective, portable, and remotely controlled surveillance system that can provide real-time monitoring without human intervention in dangerous zones. This project aims to address these challenges by developing a remotely controlled spy robot equipped with a live video streaming system, allowing for efficient surveillance and reconnaissance operations.

1.3 Problem Solution

To address the challenges of traditional surveillance systems, this project proposes the development of a Surveillance Spy Robot using an ESP32 microcontroller and an ESP32-CAM module. The robot is designed to offer real-time video monitoring, remote navigation, and wireless control, making it an ideal solution for security and surveillance applications.

The core components of the system include:

1. **ESP32 Microcontroller:** Serves as the central processing unit, handling communication and command execution.
2. **ESP32-CAM Module:** Captures and transmits real-time video feed for live surveillance.
3. **Motor Driver Circuit:** Enables precise control over the robot's movement.
4. **LM2596 Power Supply Module:** Ensures a stable power source for the components.
5. **Wi-Fi-based Android Application:** Facilitates wireless control, allowing the user to navigate the robot and monitor live video streams remotely.

The robot receives movement commands through Wi-Fi communication, processes them via the ESP32 microcontroller, and drives the motors accordingly. Simultaneously, the ESP32-CAM transmits a live video feed to the user's mobile device, enabling remote visual monitoring of the surroundings. This system provides a cost-effective and efficient solution for surveillance needs, eliminating the dependency on fixed infrastructure and enabling real-time reconnaissance in hazardous or inaccessible areas.

1.4 Key features of the Solution:

- **Real-Time Surveillance:** The ESP32-CAM module provides live video streaming for remote monitoring.
- **Wireless Control:** The robot can be controlled via a Wi-Fi-based Android application.
- **Portability:** Compact and lightweight design allows easy deployment in various environments.

- **Efficient Power Management:** The LM2596 power module ensures stable and regulated power supply.
- **Versatile Applications:** Can be used for security, reconnaissance, industrial monitoring, and disaster response.

1.5 Literature Survey

Our project focuses on designing and implementing a Surveillance Spy Robot using an ESP32, ESP32-CAM, motor driver, LM2596 power supply, and IoT -based Android control. This section aims to provide a literature survey incorporating relevant statistics and information specific to Andhra Pradesh, highlighting the potential and need for such a system in the region.

1. Security and Surveillance Market in Andhra Pradesh:

Growing Market: Andhra Pradesh is witnessing rapid urbanization and industrial development, leading to an increased demand for security and surveillance systems across residential, commercial, and industrial sectors.

Government Initiatives: The state government is actively promoting safety and security through initiatives like the establishment of specialized cyber security investigation units in each district. This indicates a growing awareness and investment in security infrastructure.

Smart City Development: With the Smart City initiative gaining momentum in India, including potential projects in Andhra Pradesh, there is a rising need for advanced surveillance solutions for urban management and citizen safety.

Key Players and Manufacturing: Andhra Pradesh is becoming a significant hub for surveillance equipment manufacturing. For instance, CP PLUS, a leading CCTV brand, has established the world's largest surveillance product manufacturing unit in the Kadapa district, indicating a strong local presence and potential for sourcing components or exploring partnerships.

Real-time Governance: The Real-time Governance Centre (RTG) in Andhra Pradesh utilizes a large network of IP cameras across 13 districts, managed by an AI-powered unified video management platform. This demonstrates the state's focus on leveraging advanced surveillance technologies for governance and security.

Increasing Investment: End-user spending on information security in India, including surveillance technologies, is projected to increase significantly. While specific figures for Andhra Pradesh are not readily available, this national trend suggests a similar growth pattern in the state.

2. Industrial Monitoring and Security in Andhra Pradesh:

Industrial Growth: Andhra Pradesh has been focusing on industrial development, with the establishment of industrial corridors and parks. This growth necessitates robust security and monitoring systems to protect assets and ensure operational safety.

Aerospace and Defence Sector: The state is emerging as a key centre for aerospace and defence manufacturing, attracting significant investments and the establishment of manufacturing units. Surveillance robots can play a crucial role in perimeter security, remote monitoring of hazardous areas, and ensuring the safety of high-value assets in these industries.

Port Security: With a significant coastline and multiple operational ports like Visakhapatnam, there is a continuous need for enhanced security measures. Surveillance robots can be valuable for patrolling large areas, monitoring critical infrastructure, and reconnaissance in port environments.

Heavy Mineral Beach Sand (HMBS) Industry: The presence of HMBS, a source for strategic minerals, necessitates secure monitoring of mining and processing sites, where a surveillance robot could be particularly useful in remote or challenging terrains.

3. Home Security in Andhra Pradesh:

Urbanization and Residential Growth: As cities in Andhra Pradesh expand, the demand for effective home security solutions is also increasing. A cost-effective and remotely controlled surveillance robot can be an attractive option for homeowners seeking to enhance their security.

Wireless and DIY Security Trends: There is a growing trend towards wireless and do-it-yourself (DIY) security solutions in India. Your project aligns with this trend by utilizing wireless Wi-Fi control and a relatively accessible ESP32 platform.

4. Defence and Military Reconnaissance in Andhra Pradesh:

Eastern Naval Command: The presence of the Eastern Naval Command in Andhra Pradesh highlights the strategic importance of the state from a defence perspective. Surveillance robots can offer a cost-effective and versatile solution for reconnaissance, perimeter security, and monitoring in naval areas.

ISRO's Launch Centre: The Satish Dhawan Space Centre (SHAR) in Srihari Kota is a critical defence and space research asset. Mobile surveillance robots could be deployed for security patrols and remote inspection tasks within the facility.

CHAPTER-2

BLOCK DIAGRAM & CIRCUIT DIAGRAM

2.1 BLOCK DIAGRAM

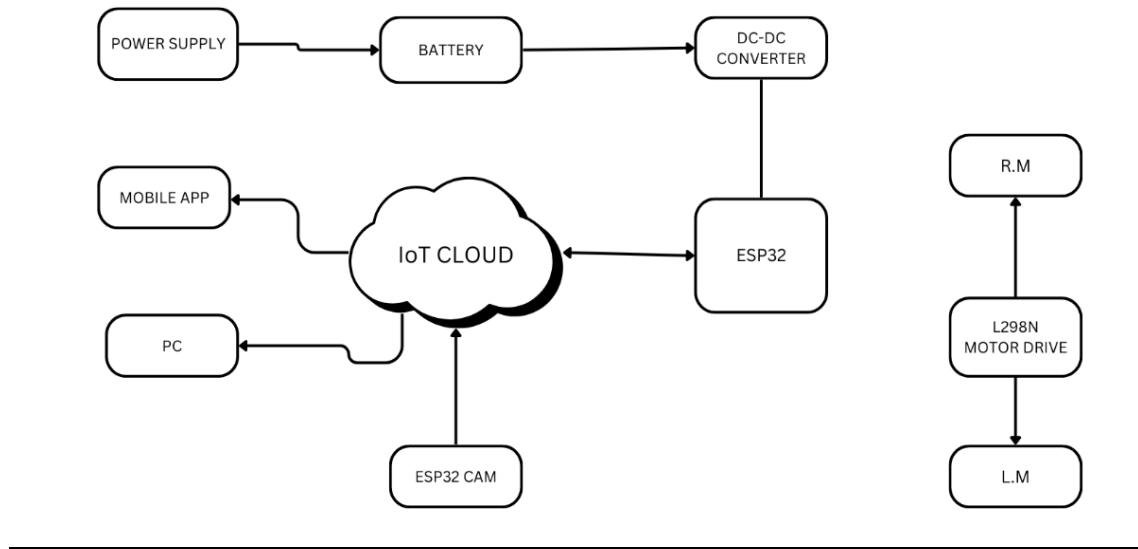


Fig: 2.1: Block Diagram

2.2 CIRCUIT DIAGRAM

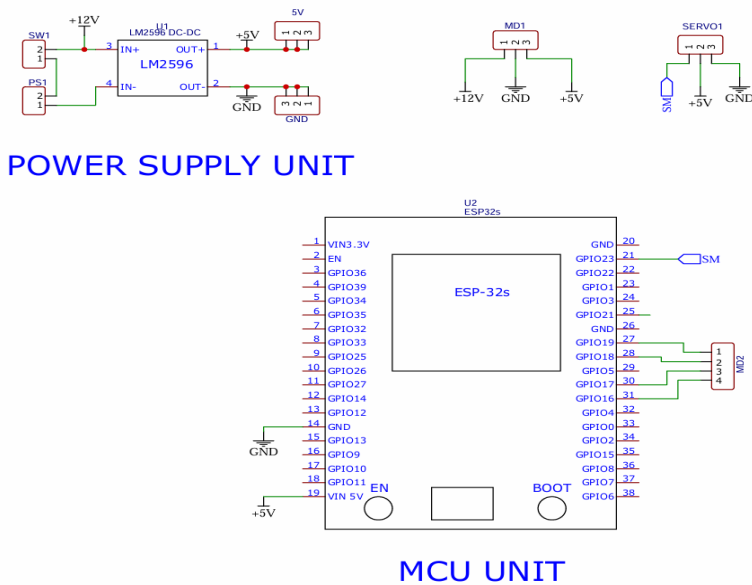


Fig: 2.2: Circuit Diagram of ESP32

CHAPTER 3

COMPONENTS & ITS SPECIFICATIONS

3.1 ESP32 Microcontroller

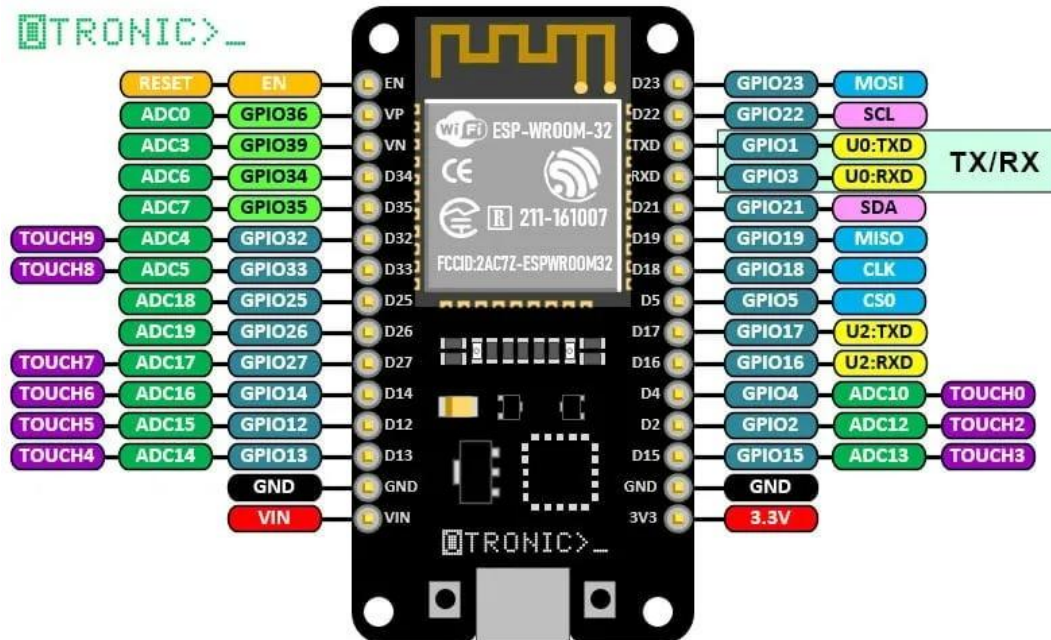


Fig 3.1: ESP32 Microcontroller

The **ESP32** microcontroller is the core processing unit of the **Surveillance Spy Robot** project. It is a low-power, high-performance **Wi-Fi and Bluetooth-enabled** microcontroller designed for IoT and embedded system applications.

Microcontroller: ESP32-WROOM-32

Operating Voltage: 3.3V

Input Voltage: 7-12V (through Vin pin)

Wi-Fi: 802.11 b/g/n

Bluetooth: BLE and Classic

GPIO Pins: 36

PWM Pins: Up to 16 channels

Flash Memory: 4MB

CPU: Dual-core Xtensa® 32-bit LX6

Clock Speed: 240 MHz

Analog Input Pins: 18 ADC channels

Communication Interfaces: UART, SPI, I2C, I2S, CAN, Ethernet

Digital Interfaces: GPIO, PWM, Touch Sensor, SDIO

Power Consumption: Low power with deep sleep mode support

3.2 ESP32 CAM



Fig 3.2: ESP32 CAM

Microcontroller: ESP32-S (32-bit)

Operating Voltage: 3.3V – 5V DC

Quiescent Current: < 10mA

Working Current: 160mA – 310mA

Camera Sensor: OV2640 (2MP)

Image Resolution: Up to 1600x1200 pixels

Wi-Fi: 802.11 b/g/n

Bluetooth: BLE and Classic

Storage: Supports microSD card (up to 4GB)

Flash Memory: 4MB

Processor Speed: Up to 240MHz

GPIO Pins: 9 usable GPIOs

Video Streaming: MJPEG, JPEG, BMP format

Communication Interfaces: UART, SPI, I2C, PWM

Trigger Input Signal: GPIO-controlled

Output Signal: Digital video stream or image data

3.3 LM2596 Power Supply Module

The **LM2596** is a step-down (buck) voltage regulator module designed to efficiently

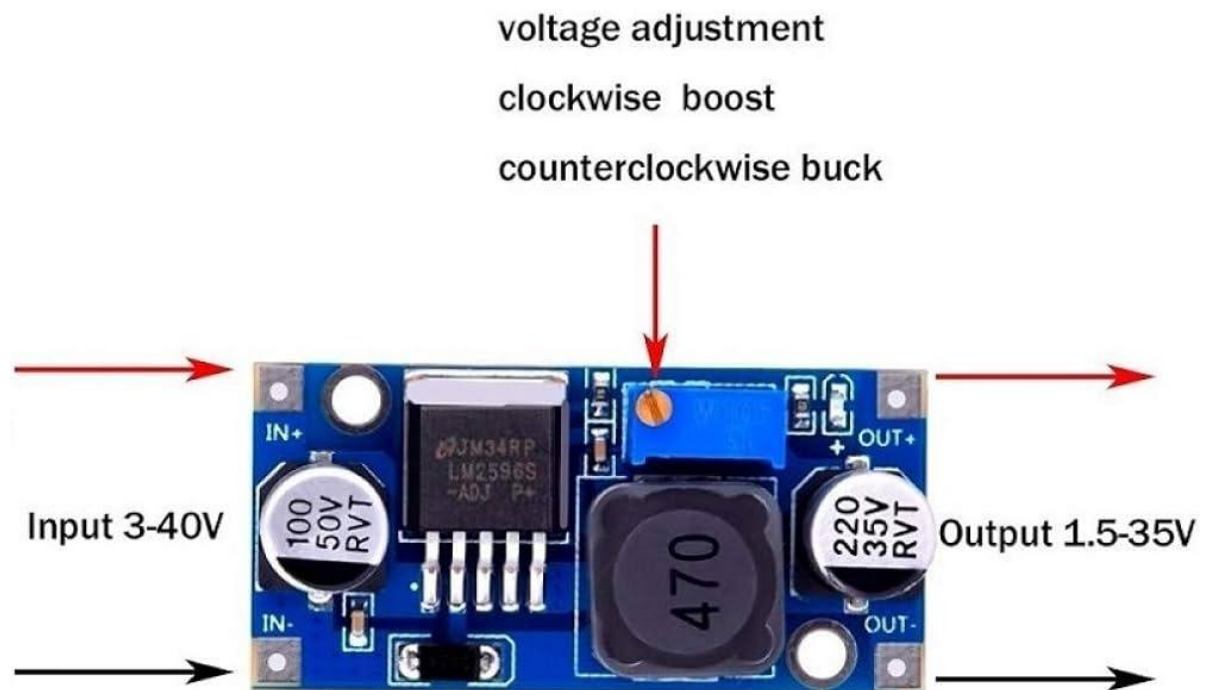


Fig 3.3: LM Power Supply Module

Specifications:

Input Voltage Range: 4.5V to 40V

Output Voltage: Adjustable from 1.23V to 37V

Maximum Output Current: 3A (with a heatsink, 2A stable without)

Efficiency: Up to 92%

Switching Frequency: 150 kHz

Load Regulation: $\pm 0.5\%$

Voltage Regulation: $\pm 2.5\%$

Thermal Shutdown Protection: Yes

Short Circuit Protection: Yes

Overload Protection: Yes

Operating Temperature Range: -40°C to $+85^{\circ}\text{C}$

Package Type: TO-220 (chip), module with onboard potentiometer for voltage adjustment

Pinout:

V_{in}: Input voltage (connects to power source, e.g., battery or adapter)

GND: Ground connection

V_{out}: Adjustable output voltage (connects to circuit components)

3.4 DC-DC Converter:

The DC-DC converter is an electromechanical device or circuitry used to convert a DC voltage from one level to another based on circuit requirements. Belonging to the electric power converter family, the DC-DC converter can be operated for small voltage applications like batteries, or high voltage applications like HV power transmission.

3.5 L298N MOTOR DRIVER:

The L298N motor driver controls both the speed and direction of rotation of a DC electric motor. It uses a L298N PWM system, which can control voltage using square wave pulses. The wider the pulses, the faster the motor will rotate. However, the exact pulse width will vary depending on the type of motor you're driving.

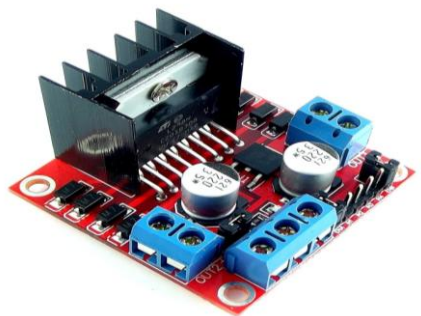


Fig:3.4: L298N Motor Driver

CHAPTER 4

SOFTWARE & INSTALLATION PROCEDURE

4.1 About Arduino IDE and its features

The Arduino IDE is an open-source software, which is used to write and upload code to the Arduino boards. The IDE application is suitable for different operating systems such as Windows, Mac OS X, and Linux. It supports the programming languages C and C++. Here, IDE stands for Integrated Development Environment. The program or code written in the Arduino IDE is often called as sketching. We can connect Arduino board / ESP32 / ESP8266 / Node MCU with the IDE to upload the sketch written in the Arduino IDE software.

The Arduino IDE will appear as in Fig 5:

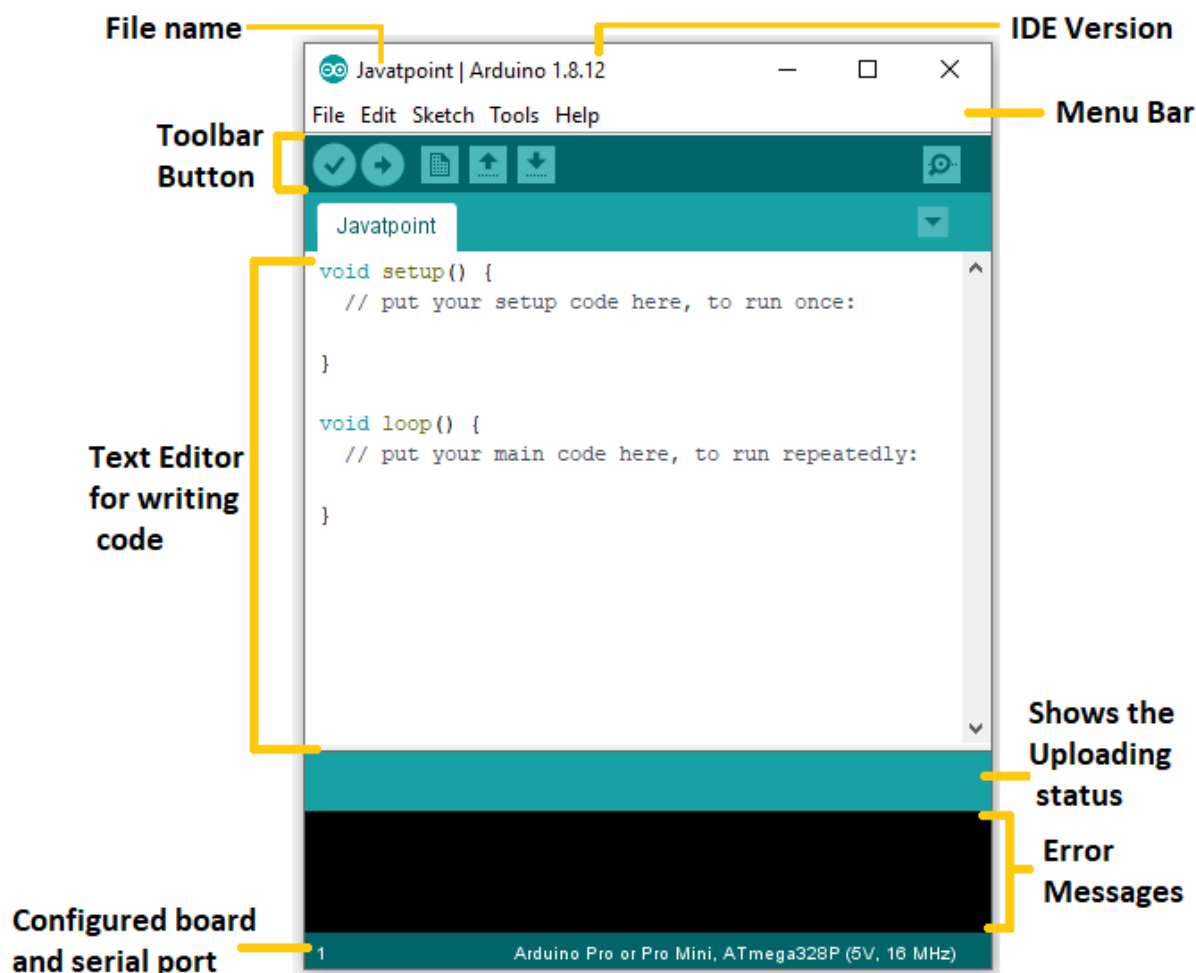


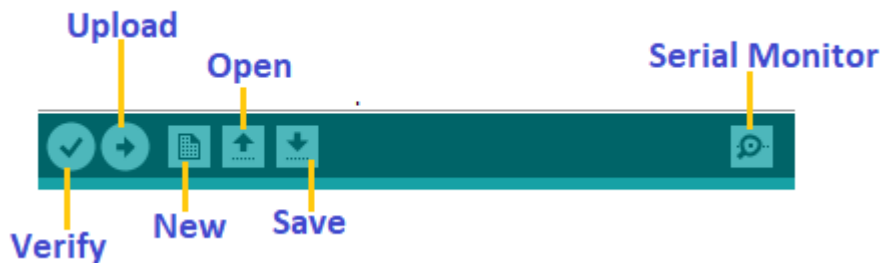
Fig 4.1: Arduino IDE Screen

Each section of the Arduino IDE display is detailed below.

Toolbar Button

The icons displayed on the toolbar are **New, Open, Save, Upload, and Verify**.

It is shown below:



Upload

The Upload button compiles and runs our code written on the screen. It further uploads the code to the connected board. Before uploading the sketch, we need to make sure that the correct board and ports are selected.

We also need a USB connection to connect the board and the computer. Once all the above measures are done, click on the Upload button present on the toolbar.

The latest Arduino boards can be reset automatically before beginning with Upload. In the older boards / certain ESP32 MCU, we need to press the Reset button present on it. As soon as the uploading is done successfully, we can notice the execution of the program example: The Blink of the LED.

If the uploading is failed, it will display the message in the error window.

We do not require any additional hardware to upload our sketch using the Arduino Bootloader.

A Bootloader is defined as a small program, which is loaded in the microcontroller present on the board.

Open

The Open button is used to open the already created file. The selected file will be opened in the current window.

Save

The save button is used to save the current sketch or code.

New

It is used to create a new sketch or opens a new window.

Verify

The Verify button is used to check the compilation error of the sketch or the written code.

Serial Monitor

The serial monitor button is present on the right corner of the toolbar. It opens the serial monitor.

It is shown below:

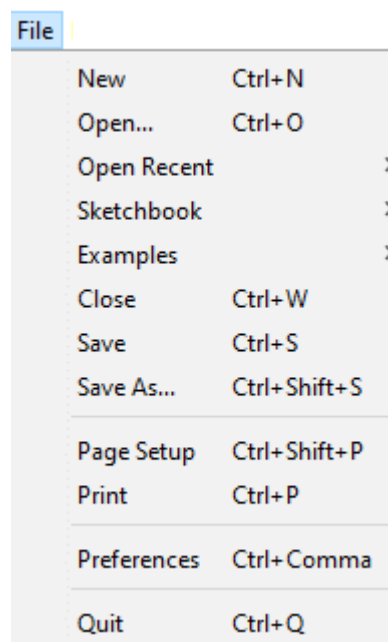


When we connect the serial monitor, the board will reset on the operating system Windows, Linux, and Mac OS X. If we want to process the control characters in our sketch, we need to use an external terminal program. The terminal program should be connected to the COM port, which will be assigned when we connect the board to the computer.

Menu Bar in IDE

File Menu

When we click on the File button on the Menu bar, a drop-down list will appear. It is shown below:



Let's discuss each option in detail.

New

The New button opens the new window. It does not remove the sketch which is already present.

Open

It allows opening the sketch, which can be browsed from the folders and computer drivers.

Open Recent

The Open Recent button contains the list of the recent sketches.

Sketchbook

It stores the current sketches created in the Arduino IDE software. It opens the selected sketch or code in a new editor at an instance.

Examples

It shows the different examples of small projects for a better understanding of the IDE and the board. The IDE provides examples of self-practice.

Close

The Close button closes the window from which the button is clicked.

Save

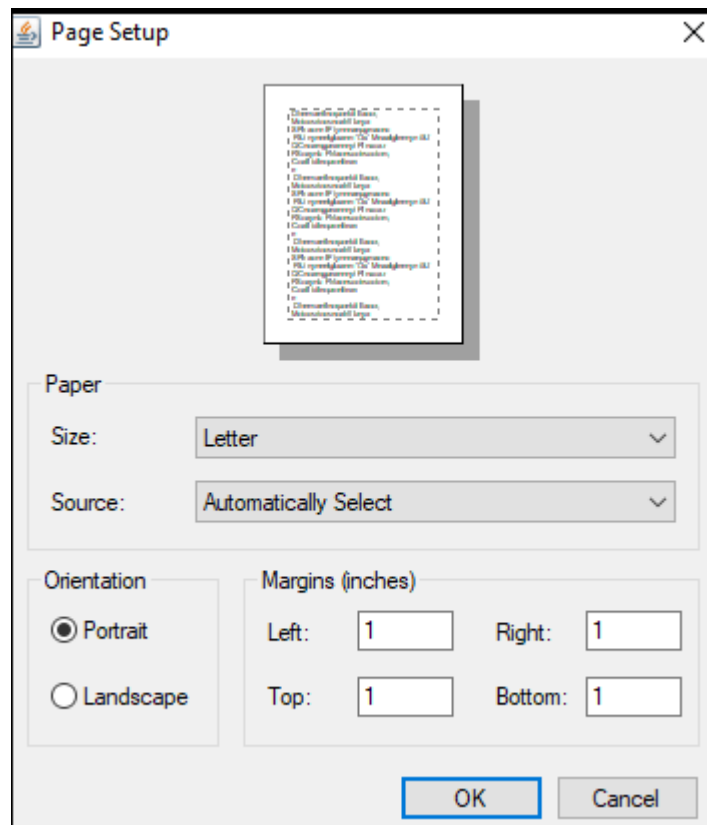
The save button is used to save the current sketch. It also saves the changes made to the current sketch. If we have not specified the name of the file, it will open the '**Save As...**' window.

Save As

We can save the sketch with a different name using the '**Save As...**' button. We can also change the name accordingly.

Page Setup

It allows setting the page margins, orientation, and size for printing. The '**Page Setup**' window will appear as:



Print

According to the settings specified in the 'Page Setup', it prepares the current sketch for printing.

Preferences

It allows the customization settings of the Arduino IDE.

Quit

The Quit button is used to close all the IDE windows. The same closed sketch will be reopened when we will open the Arduino IDE.

Edit Menu

When we click on the Edit button on the Menu bar, a drop-down list appears. It is shown below:

Edit	
Undo	Ctrl+Z
Redo	Ctrl+Y
Cut	Ctrl+X
Copy	Ctrl+C
Copy for Forum	Ctrl+Shift+C
Copy as HTML	Ctrl+Alt+C
Paste	Ctrl+V
Select All	Ctrl+A
Go to line...	Ctrl+L
Comment/Uncomment	Ctrl+Slash
Increase Indent	Tab
Decrease Indent	Shift+Tab
Increase Font Size	Ctrl+Plus
Decrease Font Size	Ctrl+Minus
Find...	Ctrl+F
Find Next	Ctrl+G
Find Previous	Ctrl+Shift+G

Let's discuss each option in detail.

Undo

The Undo button is used to reverse the last modification done to the sketch while editing.

Redo

The Redo button is used to repeat the last modification done to the sketch while editing.

Cut

It allows us to remove the selected text from the written code. The text is further placed to the clipboard. We can also paste that text anywhere in our sketch.

Copy

It creates a duplicate copy of the selected text. The text is further placed on the clipboard.

Copy for Forum

The 'Copy for Forum' button is used to copy the selected text to the clipboard, which is also suitable for posting to the forum.

Copy as HTML

The 'Copy as HTML' button is used to copy the selected text as HTML to the clipboard. It is desirable for embedding in web pages.

Paste

The Paste button is used to paste the selected text of the clipboard to the specified position of the cursor.

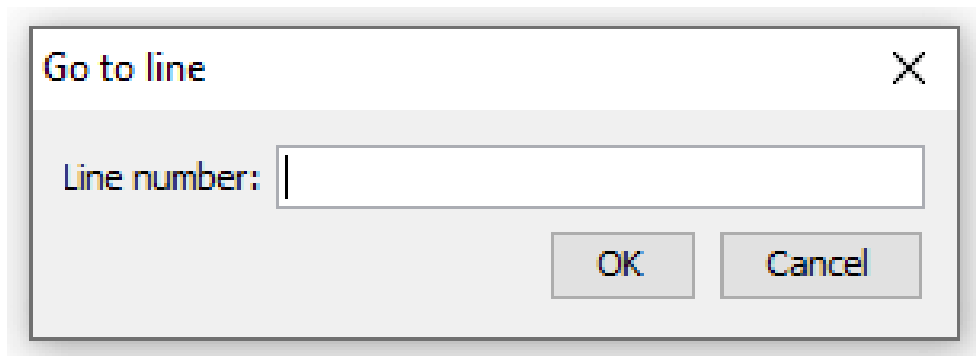
Select All

It selects all the text of the sketch.

Go to line

It moves the cursor to the specified line number.

The window will appear as:

**Comment/Decomment**

The Comment/ Decomment button is used to put or remove the comment mark (//) at the beginning of the specified line.

Increase Indent

It is used to add the space at the starting of the specified line. The spacing moves the text towards the right.

Decrease Indent

It is used to subtract or remove the space at the starting of the specified line. The spacing moves the text towards the left.

Increase Font Size

It increases the font size of the written text.

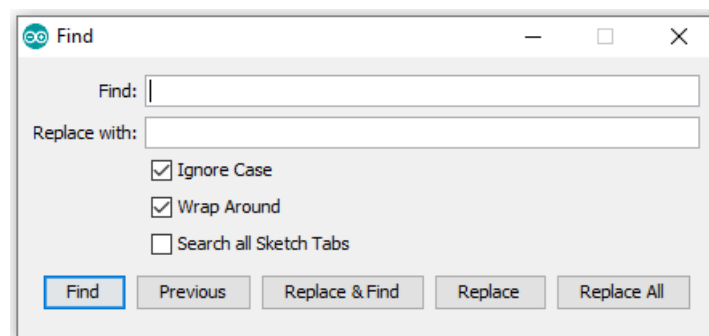
Decrease Font Size

It decreases the font size of the written text.

Find

It is used to find the specified text. We can also replace the text. It highlights the text in the sketch.

The window will appear as:



Find Next

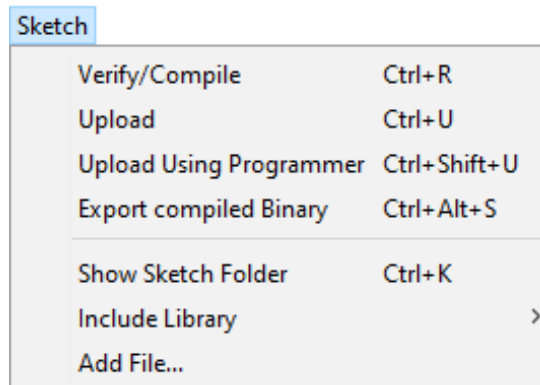
It highlights the next word, which has specified in the '**Find...**' window. If there is no such word, it will not show any highlighted text.

Find Previous

It highlights the previous word, which has specified in the '**Find**' window. If there is no such word, it will not show any highlighted text.

Sketch Menu

When we click on the Sketch button on the Menu bar, a drop-down list appears. It is shown below:



Let's discuss each option in detail.

Verify/Compile

It will check for the errors in the code while compiling. The memory in the console area is also reported by the IDE.

Upload

The Upload button is used to configure the code to the specified board through the port.

Upload Using Programmer

It is used to override the Bootloader that is present on the board. We can utilize the full capacity of the Flash memory using the '**Upload Using Programmer**' option. To implement this, we need to restore the Bootloader using the **Tools-> Burn Bootloader** option to upload it to the USB serial port.

Export compiled Binary

It allows saving a **.hex** file and can be kept archived. Using other tools, **.hex** file can also be sent to the board.

Show Sketch Folder

It opens the folder of the current code written or sketch.

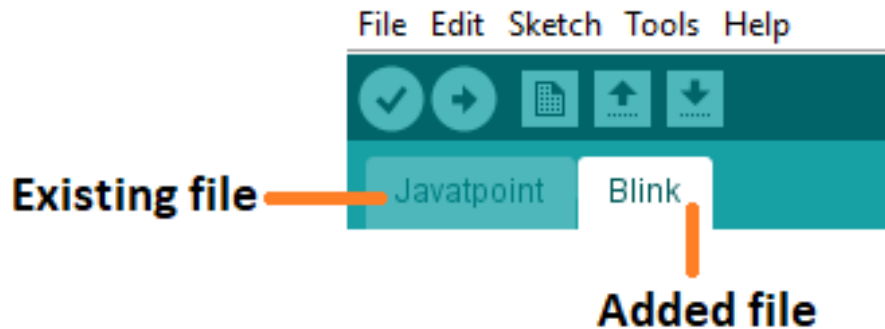
Include Library

Include Library includes various Arduino libraries. The libraries are inserted into our code at the beginning of the code starting with the **#**. We can also import the libraries from **.zip** file.

Add File...

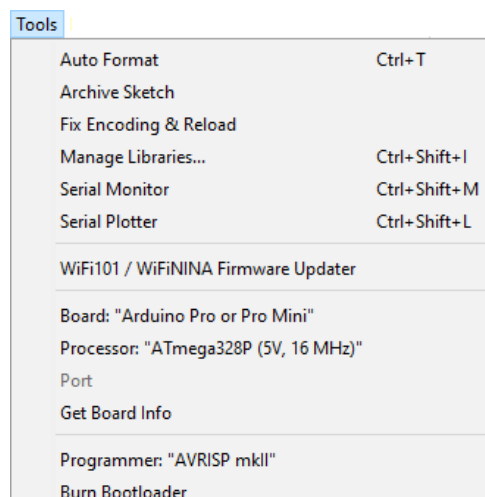
The Add File... button is used to add the created file in a new tab on the existing file.

For example, let's add '**Blink**' file to the '**Javatpoint**' file. The tab will now appear as:



Tools Menu

When we click on the Tools button on the Menu bar, a drop-down list appears. It is shown below:



Let's discuss each option in detail.

Auto Format

The Auto Format button is used to format the written code. For example, lining the open and closed curly brackets in the code.

Archive Sketch

The copy of the current sketch or code is archived in the .zip format. The directory of the archived is same as the sketch.

Fix Encoding and Reload

This button is used to fix the inconsistency between the operating system char maps and editor char map encoding.

Manage Libraries...

It shows the updated list of all the installed libraries. We can also use this option to install a new library into the Arduino IDE.

Serial Monitor

It allows the exchange of data with the connected board on the port.

Serial Plotter

The Serial Plotter button is used to display the serial data in a plot. It comes preinstalled in the Arduino IDE.

Wi-Fi 101/Wi-Fi NINA Firmware Updater

It is used to check and update the Wi-Fi Firmware of the connected board.

Board

We are required to select the board from the list of boards. The selected board must be similar to the board connected to the computer.

Processor

It displays the processor according to the selected board. It refreshes every time during the selection of the board.

Port

It consists of the virtual and real serial devices present on our machine.

Get Board Info

It gives the information about the selected board. We need to select the appropriate port before getting information about the board.

Programmer

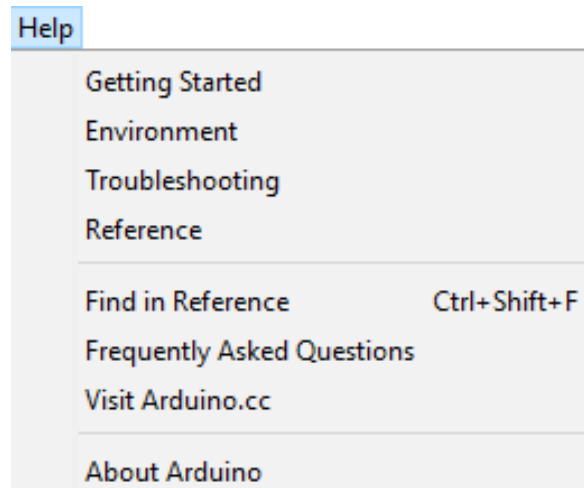
We need to select the hardware programmer while programming the board. It is required when we are not using the onboard USB serial connection. It is also required during the burning of the Bootloader.

Burn Bootloader

The Bootloader is present on the board onto the microcontroller. The option is useful when we have purchased the microcontroller without the bootloader. Before burning the bootloader, we need to make sure about the correct selected board and port.

Help Menu

When we click on the Help button on the Menu bar, a drop-down list will appear. It is shown below:



The Help section includes several documents that are easy to access, which comes along with the Arduino IDE. It consists of the number of options such as Getting Started, Environment, Troubleshooting, Reference, etc. can be accessed without the internet connection as well. It will directly link us to the official website of Arduino.

4.2 Arduino IDE

1. Easy to use: Arduino IDE provides a user-friendly interface for writing, compiling, and uploading code.
2. Cross-platform: Available for Windows, macOS, and Linux.
3. Open source: Free to download, modify, and distribute.

4. Large community: Extensive libraries, tutorials, and forums.
5. Versatile: Supports various microcontrollers, including Arduino, ESP32, and ESP8266.

4.3 Arduino IDE for ESP32/ESP8266

1. Simplified development: Arduino IDE streamlines the development process for ESP32/ESP8266.
2. Library support: Access to a vast library collection, including Wi-Fi, Bluetooth, and IoT-specific libraries.
3. Community resources: Leverage ESP32/ESP8266-specific tutorials, examples, and forums.
4. Easy uploading: Upload code directly from Arduino IDE to ESP32/ESP8266 boards.
5. Cost-effective: No need for expensive development tools or software.

ESP32-specific benefits:

1. Dual-core processing: Arduino IDE supports ESP32's dual-core architecture.
2. Wi-Fi and Bluetooth: Easily integrate Wi-Fi and Bluetooth capabilities.
3. Low-power consumption: Optimize power usage with Arduino IDE's built-in features.
4. Advanced peripherals: Access ESP32's advanced peripherals, such as SPI, I2C, and UART.

ESP8266-specific benefits:

1. Wi-Fi integration: Easily integrate Wi-Fi capabilities.
2. Low-cost: ESP8266 is an affordable board.
3. Small footprint: Ideal for IoT projects requiring compact design.
4. Community support: Extensive ESP8266 community and resources.

Comparison with other IDEs:

1. Platform IO: More complex but offers advanced features and multi-platform support.
2. Visual Studio Code: More versatile but requires additional setup and configuration.
3. ESP-IDF: Official ESP32/ESP8266 IDE, but steeper learning curve.

Arduino IDE provides an ideal balance of ease, versatility, and community support, making it a popular choice for ESP32 and ESP8266 development.

4.4 Step by Step Installation Procedure

In this section, we will learn in easy steps, how to set up the Arduino IDE on our computer and prepare the board to receive the program via USB cable.

The Arduino IDE 2 is an open-source project. It is a big step from its sturdy predecessor, Arduino IDE 1.x, and comes with revamped UI, improved board & library manager, debugger, autocomplete feature and much more.

Step 1 – Download and install the Arduino IDE 2 on your Windows. You can easily download the editor by visit the Arduino Software page: <https://www.arduino.cc/en/software>

Requirements

- Windows - Win 10 and newer, 64 bits
- Linux - 64 bits
- macOS - Version 10.15: "Catalina" or newer, 64 bits

For windows installation: In Arduino IDE 2.3.3 select Win 10 and newer, 64 bits



Fig 6. Arduino Software page

To install the Arduino IDE 2 on a Windows computer, simply run the file downloaded from the software page.

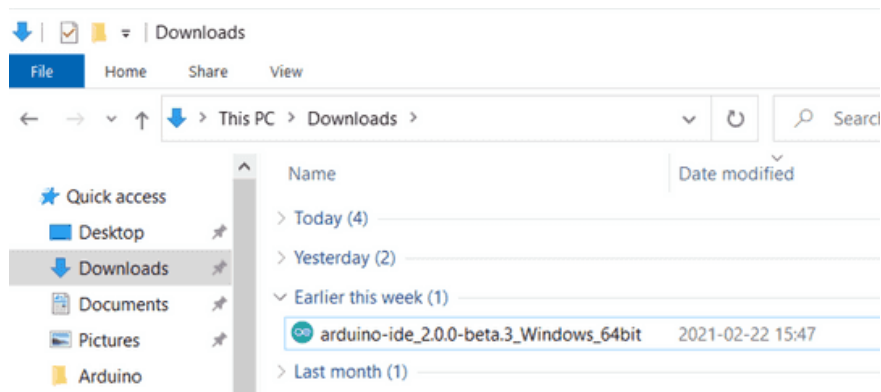
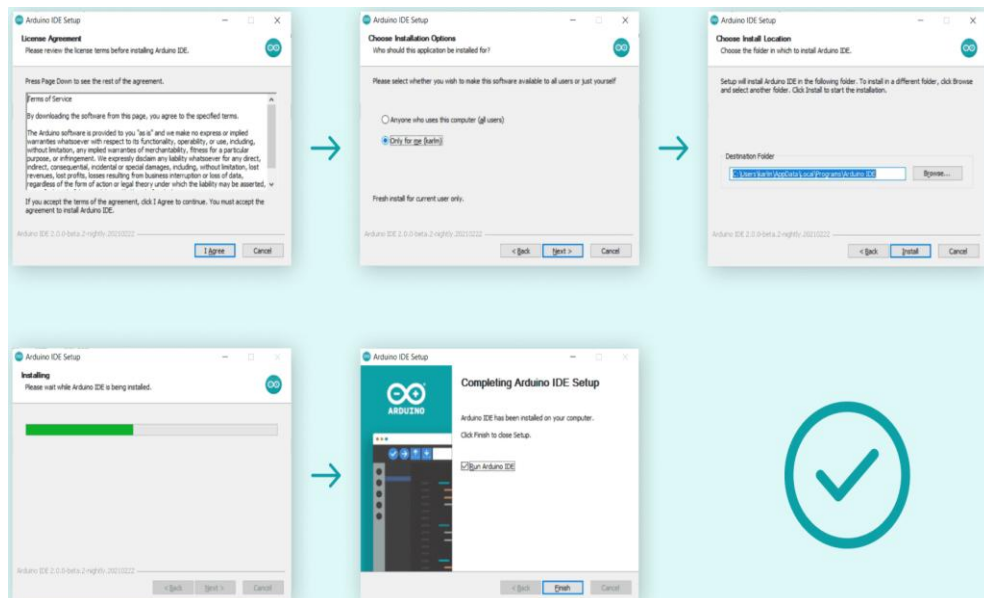


Fig 7. Running the Executable Arduino IDE 2 installation file

Follow the instructions in the installation guide as shown in the image. The installation may take several minutes.



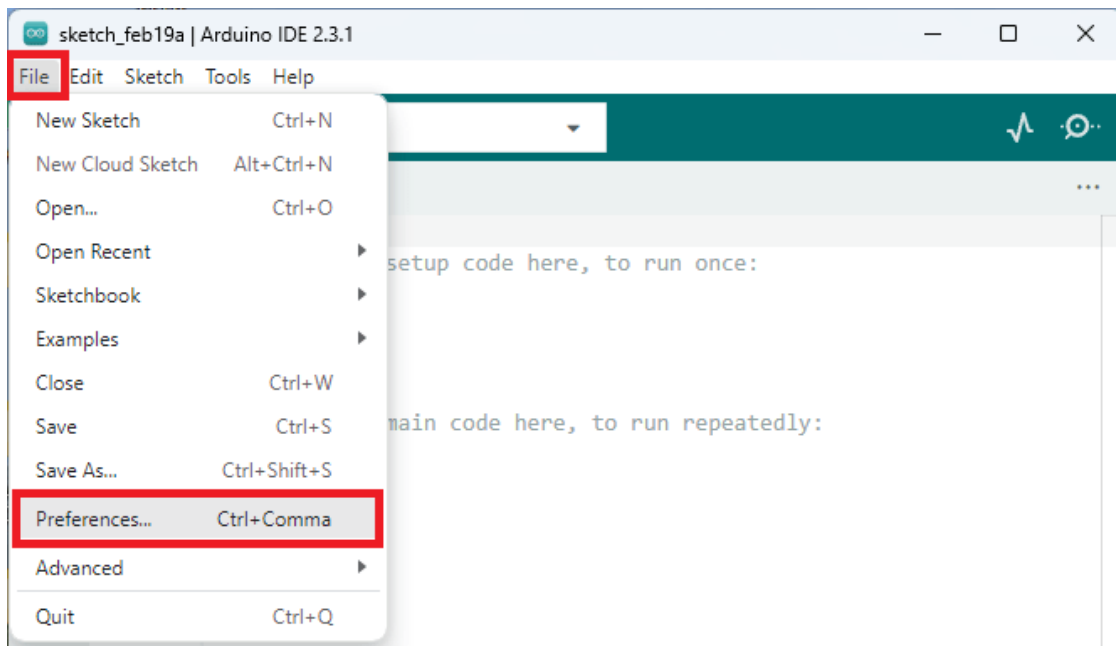
Installing ESP32 Board in Arduino IDE 2

Before proceeding make sure you have completed above process that is Arduino IDE 2 installed on your computer and follow the instructions given below

Install ESP32 Add-on in Arduino IDE

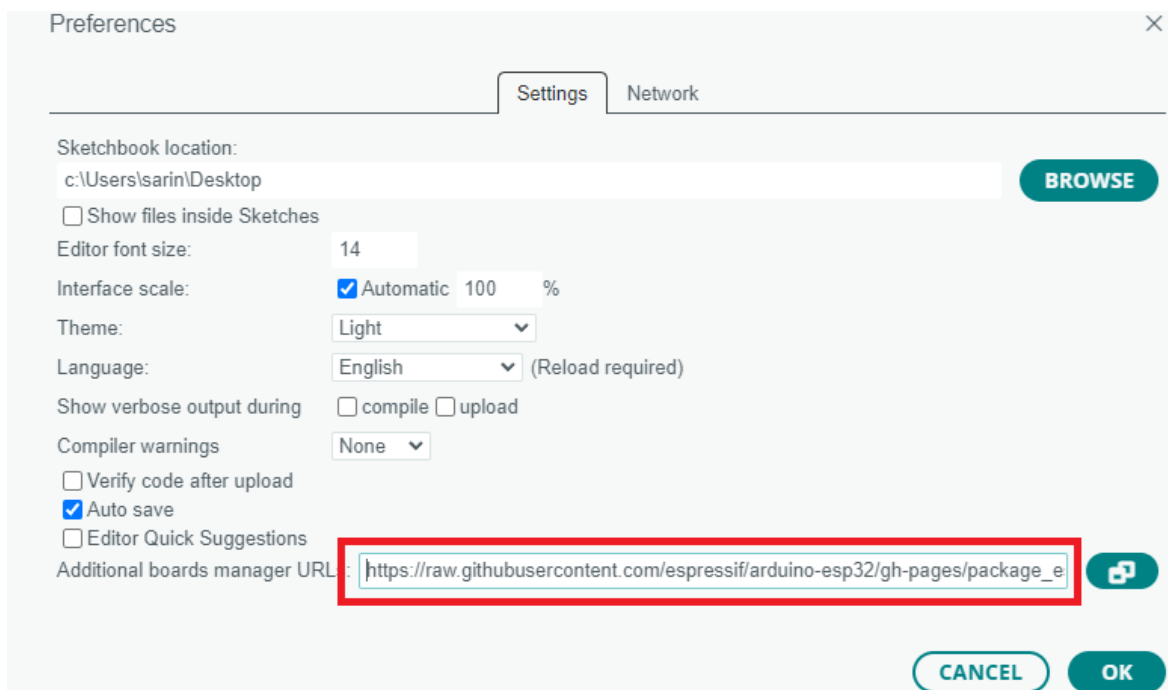
To install the ESP32 board in your Arduino IDE, follow these next instructions:

1. In your Arduino IDE 2, go to **File > Preferences**.



2. Copy and paste the following line to the **Additional Boards Manager** URLs field.

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json



Note: if you already have the ESP8266 boards URL, you can separate the URLs with a comma, as follows:

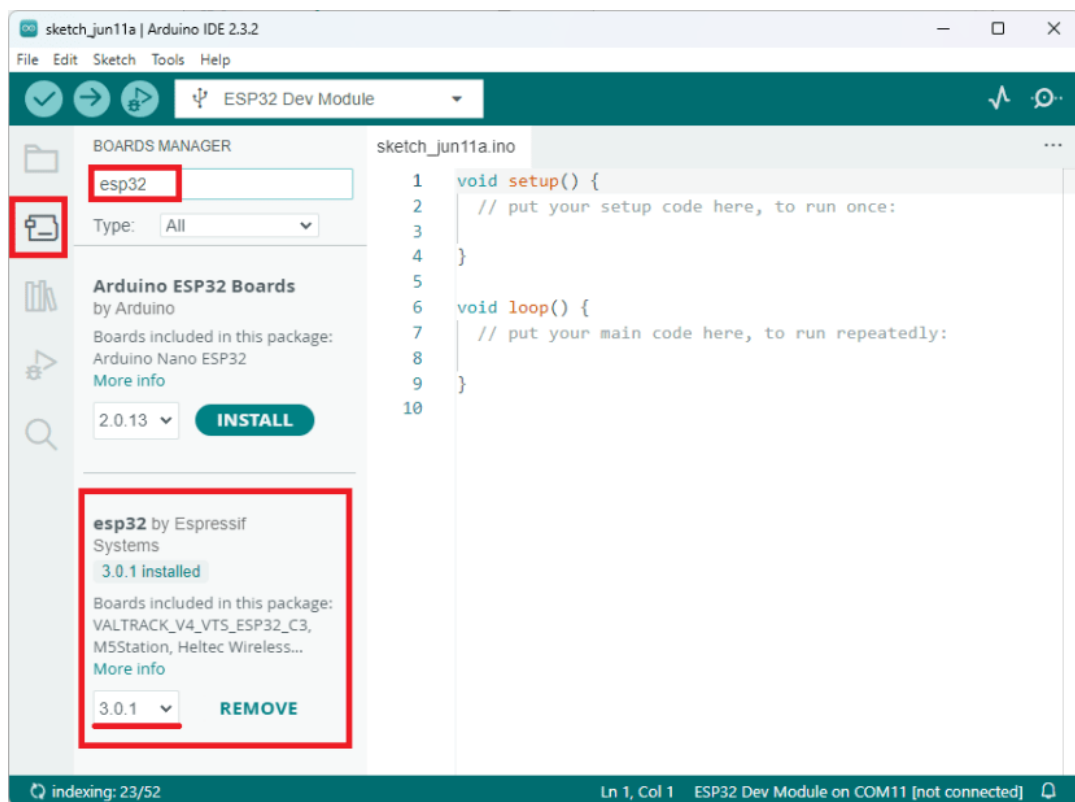
http://arduino.esp8266.com/stable/package_esp8266com_index.json,

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

3. Open the Boards Manager. You can go to **Tools > Board > Boards Manager...** or you can simply click the **Boards Manager** icon in the left-side corner.



4. Search for **ESP32** and press install button for **esp32 by Espressif Systems version 3.X**.



That's it. It should be installed after a few seconds.

Testing the Installation

To test the ESP32 add-on installation, upload a simple code that blinks the on-board LED (GPIO 2).

Write the following code to your Arduino IDE:

```
#include <Arduino.h>

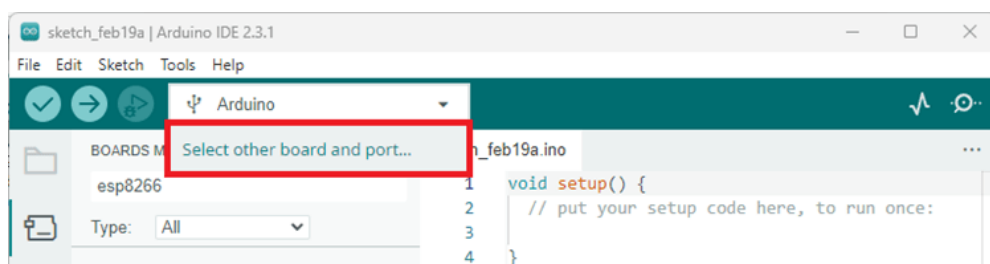
#define LED 2

void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  pinMode(LED, OUTPUT);
}

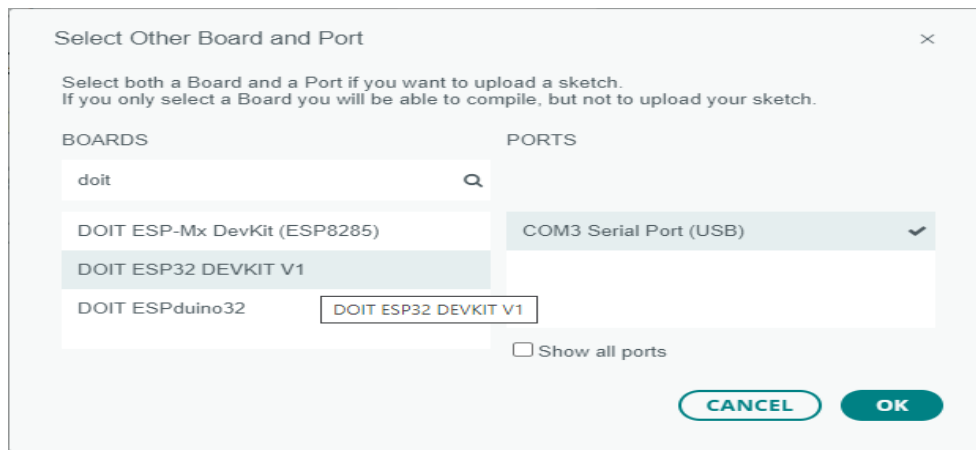
void loop() {
  // put your main code here, to run repeatedly:
  digitalWrite(LED, HIGH);
  Serial.println("LED is on");
  delay(1000);
  digitalWrite(LED, LOW);
  Serial.println("LED is off");
  delay(1000);
}
```

Uploading the Sketch

Select your board before uploading the code. On the top drop-down menu, click on “**Select other board and port...**”



A new window, as shown below, will open. Search for your ESP32 board model.



Select the ESP32 board model you're using, and the COM port. In our example, we're using the DOIT ESP32 DEVKIT V1. Click **OK** when you're done.

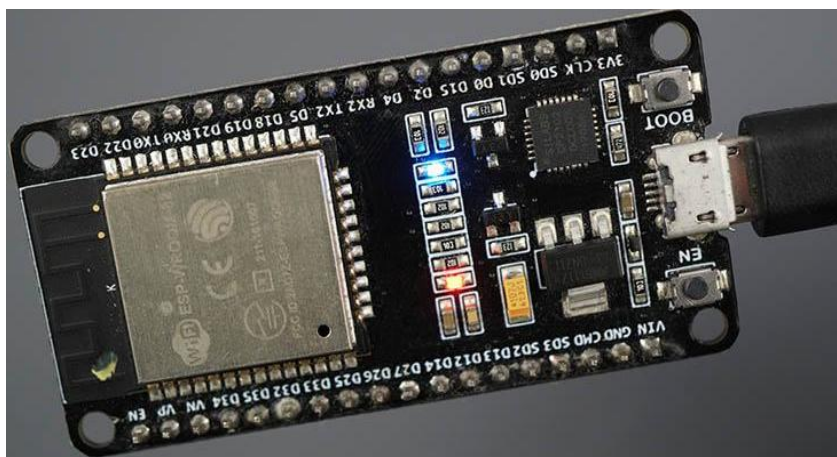
Now, you just need to click on the **Upload** button shown below.



After a few seconds, the upload should be complete.

Note: some ESP32 development boards don't go into flashing/uploading mode automatically when uploading a new code and you'll see a lot of dots on the debugging window followed by an error message. If that's the case, you need to press the ESP32 BOOT button when you start seeing the dots on the debugging window.

The ESP32 on-board LED should be blinking every second.



Serial Monitor

You can click on the Serial Monitor icon to open the Serial Monitor tab. Make sure you select the 115200 baud rate.



That's it! You've installed the ESP32 Boards successfully in Arduino IDE 2.

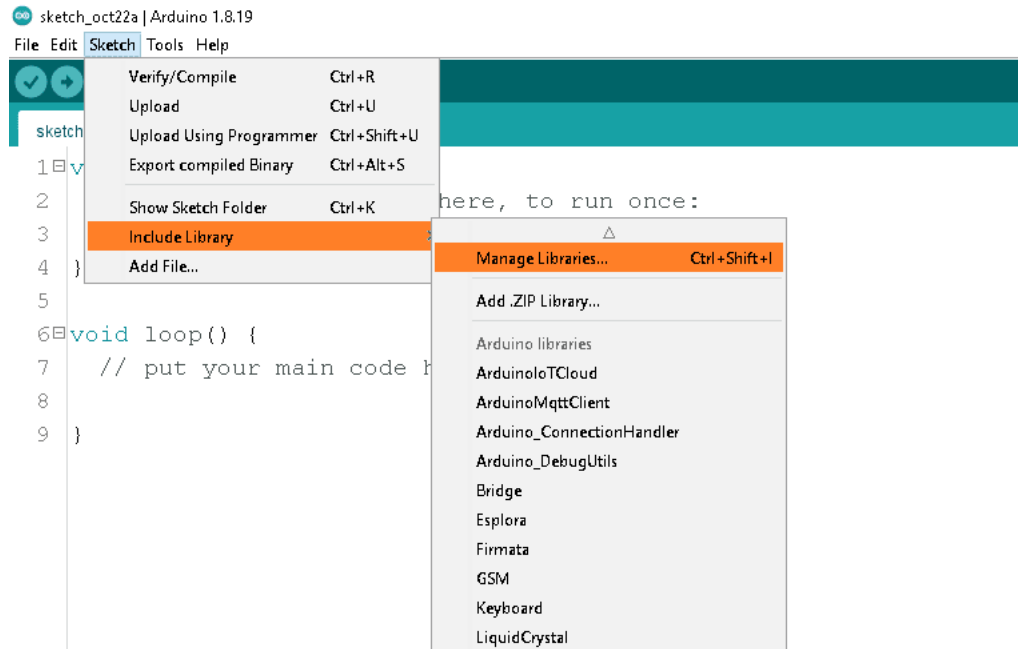
Troubleshooting

If you try to upload a new sketch to your ESP32 and you get this error message “A fatal error occurred: Failed to connect to ESP32: Timed out... Connecting...”. It means that your ESP32 is not in flashing/uploading mode.

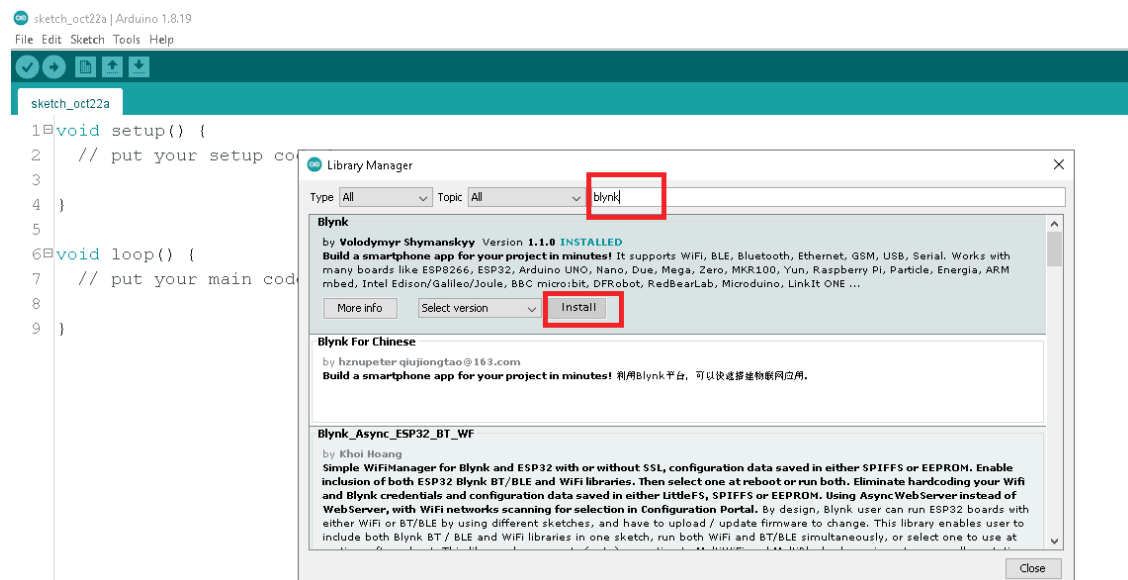
- Having the right board name and COM port selected, follow these steps:
- Hold-down the **BOOT** button in your ESP32 board
- Press the **Upload** button in the Arduino IDE to upload your sketch
- After you see the “Connecting....” message in your Arduino IDE, release the finger from the **BOOT** button
- After that, you should see the “Done uploading” message

Blynk Library Installation

Install a new library Blynk into your Arduino IDE, you can use the Library Manager. First, connect your computer to the Internet. Open the IDE and click to the “Sketch” menu and then Include Library > Manage Libraries.



Then the Library Manager will open and you will find a list of libraries that are already installed or ready for installation. Search for Blynk library and in the version, selection choose the latest version to date.



Finally click on Install and wait for the IDE to install the new library. Downloading may take time depending on your connection speed. Once it has finished, an Installed tag should appear next to the Bridge library. You can close the library manager.

CHAPTER 5

IOT CLOUD & SERVER PLATFORM

Internet of Things (IoT)

The internet of things, or IoT, is a network of interrelated devices that connect and exchange data with other IoT devices and the cloud. IoT devices are typically embedded with technology such as sensors and software and can include mechanical and digital machines and consumer objects. These devices encompass everything from everyday household items to complex industrial tools. Increasingly, organizations in a variety of industries are using IoT to operate more efficiently, deliver enhanced customer service, improve decision-making and increase the value of the business. With IoT, data is transferable over a network without requiring human-to-human or human-to-computer interactions. A thing in the internet of things can be a person with a heart monitor implant, a farm animal with a biochip transponder, an automobile that has built-in sensors to alert the driver when tire pressure is low, or any other natural or man-made object that can be assigned an Internet Protocol address and can transfer data over a network.

How does IoT work

IoT systems function by gathering data from sensors embedded in IoT devices, which is then transmitted through an IoT gateway for analysis by an application or back-end system

Sensors or devices

An IoT ecosystem consists of web-enabled smart devices that use embedded systems, such as processors, sensors and communication hardware to collect, send and act on data acquired from their environments.

Connectivity

IoT devices can communicate with one another through a network over the internet. These devices share sensor data by connecting to an IoT gateway, which acts as a central hub where IoT devices can send data. Before the data is shared, it can also be sent to an edge device where it is analyzed locally.

Data analysis

Only the relevant data is used to identify patterns, offer recommendations and identify potential issues before they escalate. Analyzing data locally reduces the volume of data sent to the cloud, which minimizes bandwidth consumption. Sometimes, these devices communicate with other related devices and act on the information they get from one another. The devices do

most of the work without human intervention, although people can interact with the devices. For example, they can set them up, give them instructions or access the data. The connectivity, networking and communication protocols used with these web-enabled devices largely depend on the specific IoT applications deployed. IoT can also use artificial intelligence and machine learning to make data collection processes easier and more dynamic.

Graphical user interface

A graphical user interface (UI) is typically used to manage IoT devices. For example, a website or a mobile app can be used as an UI to manage, control and register smart devices.



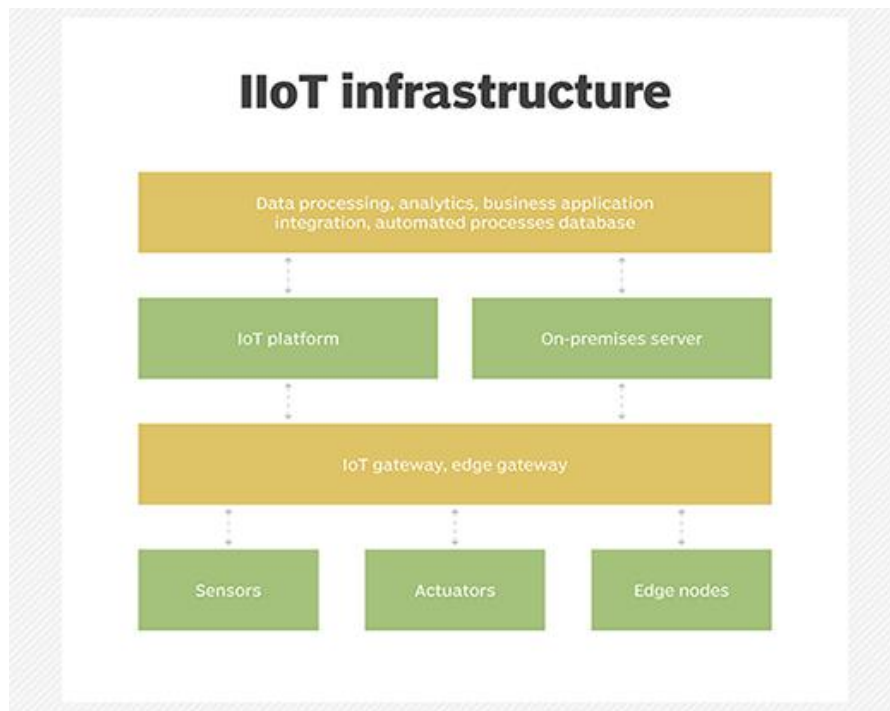
Consumer, enterprise and industrial IoT devices include smart TVs and smart sensors outfitted for conference rooms and assembly line machines.

IoT applications include the following:

- **Agriculture.** IoT can benefit farmers by making their jobs easier. For example, sensors can collect data on rainfall, humidity, temperature and soil content and IoT can help automate farming techniques. Additionally, IoT devices can be used to oversee the health of livestock, monitor equipment and streamline supply chain management.
- **Construction.** IoT can help monitor operations surrounding infrastructure. Sensors, for example, can monitor events or changes within structural buildings, bridges and other infrastructure that could potentially compromise safety. This provides benefits such as improved incident management and response, reduced operations costs and improved service quality.
- **Home automation.** A home automation business can use IoT to monitor and manipulate mechanical and electrical systems in a building. Homeowners can also remotely control and automate their home environment by using IoT devices, including smart thermostats,

lighting systems, security cameras and voice assistants such as Alexa and Siri for increased comfort and energy efficiency.

- **Smart buildings and cities.** Smart cities can help citizens reduce waste and energy consumption. They can reduce energy costs using sensors that detect how many occupants are in a room and turning the air conditioner on if sensors detect a conference room is full or lowering the heat if everyone in the office has gone home.
- **Urban consumption systems.** IoT technologies can also be used to monitor and manage urban consumption such as traffic lights, parking meters, waste management systems and public transportation networks.
- **Healthcare monitoring.** IoT devices such as remote patient monitoring systems, smart medical devices and medication trackers let healthcare providers monitor patients' health status, manage chronic conditions and provide timely interventions. IoT gives providers the ability to monitor patients more closely by analyzing the generated data. Hospitals also often use IoT systems to complete tasks such as inventory management for both pharmaceuticals and medical instruments.
- **Retail.** IoT sensors and beacons in retail stores can track customer movement, analyze shopping patterns, manage inventory levels and personalize marketing messages. This enhances the shopping experience for customers and optimizes store operations.
- **Transportation.** IoT devices help the transportation industry by monitoring vehicle performance, optimizing routes and tracking shipments. For example, the fuel efficiency of connected cars can be monitored to reduce fuel costs and improve sustainability. IoT devices can also monitor the condition of cargo, so it reaches its destination in optimal condition.
- **Wearable devices.** Wearable devices with sensors and software can collect and analyze user data, sending messages to other technologies about the users to make their lives easier and more comfortable. Wearable devices are also used for public safety -- for example, by improving first responders' response times during emergencies by providing optimized routes to a location or by tracking construction workers' or firefighters' vital signs at life-threatening sites.
- **Energy management.** IoT-enabled smart grids, smart meters and energy management systems let utility companies and consumers monitor and optimize energy usage, manage demand-response programs and integrate renewable energy sources more efficiently. For



IoT Cloud

An IoT cloud is a massive network that supports IoT devices and applications. This includes the underlying infrastructure, servers and storage, needed for real-time operations and processing. An IoT cloud also includes the services and standards necessary for connecting, managing, and securing different IoT devices and applications.

Servers

Servers in IoT connect physical devices to the internet, allowing them to communicate and share data. They can be used to monitor and control devices remotely, and to allow devices to interact with each other autonomously.

Here are some types of servers used in IoT:

- **IoT Data Server**

A data integration controller that can be used in factories, production lines, and cloud systems. It has standard data management functions for collection, processing, saving, noticing, and publishing.

- **IoT web server**

A platform that connects devices to the internet, allowing them to communicate and share data. It can be used to monitor and control devices remotely, and to allow devices to interact with each other autonomously.

- **Windows Server IoT**

A full version of Windows Server that provides enterprise manageability and security to IoT solutions.

- **Embedded web server**

A small, lightweight web server that can be integrated into other software or hardware systems. It's used to provide a web-based interface for monitoring and controlling devices.

- **Network server**

A server that plays a crucial role in IoT ecosystems by managing network resources and device communications. It allows IoT networks to scale up to support thousands or even millions of devices.

Message Queue Telemetry Transport Protocol (MQTT)

Message Queuing Telemetry Transport, or MQTT, is a communications protocol designed for Internet of Things devices with extremely high latency and restricted low bandwidth. Message Queuing Telemetry Transport is a perfect protocol for machine-to-machine (M2M) communication since it is designed specifically for low-bandwidth, high-latency settings.

Message Queue Telemetry Transport Protocol (MQTT)

MQTT is a simple, lightweight messaging protocol used to establish communication between multiple devices. It is a TCP-based protocol relying on the publish-subscribe model. This communication protocol is suitable for transmitting data between resource-constrained devices having low bandwidth and low power requirements. Hence this messaging protocol is widely used for communication in the IoT Framework.

Publish-Subscribe Model

This model involves multiple clients interacting with each other, without having any direct connection established between them. All clients communicate with other clients only via a third party known as a Broker.

MQTT Client and Broker

Clients publish messages on different topics to brokers. The broker is the central server that receives these messages and filters them based on their topics. It then sends these messages to respective clients that have subscribed to those different topics. The heart of any publish/subscribe protocol is the MQTT broker. A broker can handle up to thousands of concurrently connected MQTT customers, depending on how it is implemented. All communications must be received by the broker, who will then sort them, ascertain who subscribed to each one, and deliver the messages to the clients who have subscribed. All persistent customers' sessions, including missed messages and subscriptions, are likewise kept by the Broker.

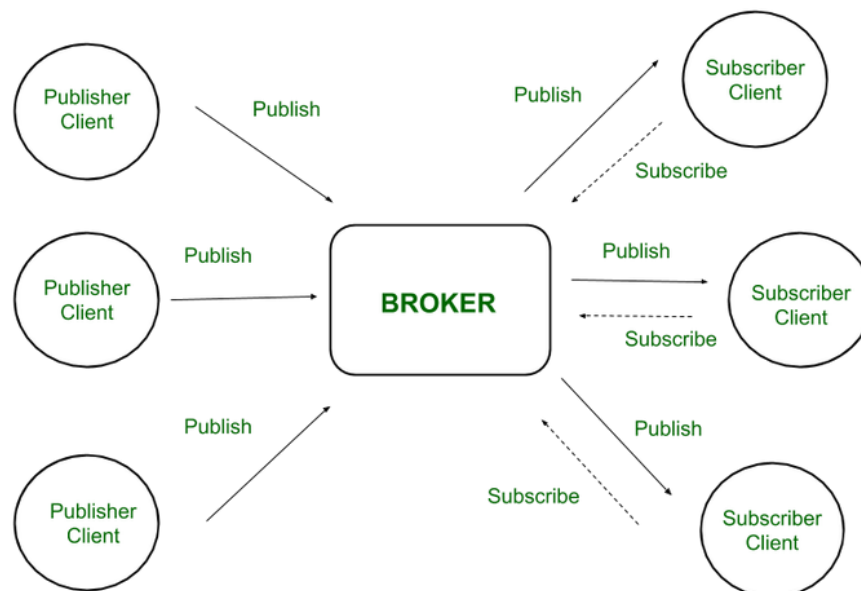


Fig. 5.1: Publish-Subscribe Model

Hence client that has subscribed to a specific topic receives all messages published on that topic. Here the broker is central hub that receives messages, filters them, and distributes them to appropriate clients, such that both message publishers, as well as subscribers, are clients.

Working of MQTT

MQTT's publish/subscribe (pub/sub) communication style, which aims to maximize available bandwidth, is an alternative to conventional client-server architecture that communicates directly with an endpoint. In contrast, the client who transmits the message (the publisher) and the client or clients who receive it (the subscribers) are not connected in the

pub/sub paradigm. Third parties—the brokers—manage the relationships between the publishers and subscribers because they don't communicate with one another directly.

Publishers and subscribers, which denote whether a client is publishing messages or has subscribed to receive messages, are examples of MQTT clients. The same MQTT client can be used to accomplish these two features. A publish occurs when a client or device want to submit data to a server or broker.

The term “subscribe” refers to the reversal of the procedure. Several clients can connect to a broker under the pub/sub paradigm and subscribe to subjects that interest them.



When a broker and a subscribing client lose contact, the broker will store messages in a buffer and send them to the subscriber whenever the broker is back up and running. The broker has the right to cut off communication with subscribers and send them a cached message containing publisher instructions if the publishing client abruptly disconnects from the broker. “Publishers send the messages, subscribers receive the messages they are interested in, and brokers pass the messages from the publishers to the subscribers,” reads an IBM write-up describing the pub/sub paradigm. MQTT clients, such as publishers and subscribers, can only speak with MQTT brokers. Any device or programme that runs a MQTT library can be a MQTT client, ranging from microcontrollers like the Arduino to entire application servers housed in the cloud.

Characteristics of MQTT

- **Lightweight:** MQTT is designed to be lightweight, making it suitable for use in aid-restrained environments inclusive of embedded systems and low-strength devices. The protocol minimizes bandwidth and processing overhead, enabling green communication even on restricted networks.
- **Publish-Subscribe Model:** In the publish-subscribe version, clients (publishers) send messages to subjects, and different clients (subscribers) acquire messages from subjects

of interest. This decoupling of producers and purchasers permits for flexible and dynamic conversation styles.

- **Quality of Service (QoS) Levels:** MQTT supports exclusive stages of message delivery warranty, referred to as Quality of Service (QoS). QoS levels range from 0 to 2, providing various stages of reliability and message transport guarantees, relying at the utility necessities.
- **Retained Messages:** MQTT lets in agents to store retained messages on topics, making sure that new subscribers acquire the maximum latest message posted on a subject right now after subscribing. This characteristic is beneficial for fame updates and configuration settings.
- **Last Will and Testament (LWT):** MQTT clients can specify a Last Will and Testament message to be posted by way of the broker in the occasion of a sudden consumer disconnect. This function affords a mechanism for detecting patron failures and dealing with them gracefully.
- **Security:** MQTT helps various protection mechanisms, consisting of Transport Layer Security (TLS) encryption and authentication mechanisms which include username/password and consumer certificates. These capabilities make certain the confidentiality, integrity, and authenticity of messages exchanged over MQTT connections.

Advantages of MQTT

This model is not restricted to one-to-one communication between clients. Although the publisher client sends a single message on specific topic, broker sends multiple messages to all different clients subscribed to that topic. Similarly, messages sent by multiple such publisher clients on multiple different topics will be sent to all multiple clients subscribed to those topics. Hence one-to-many, many-to-one, as well as many-to-many communication is possible using this model. Also, clients can publish data and at the same time receive data due to this two-way communication protocol. Hence MQTT is considered to be bi-directional protocol. The default unencrypted MQTT port used for data transmission is 1883. The encrypted port for secure transmission is 8883.

- Lightweight protocol that is quick to create and allows for efficient data transport
- Minimal data packet usage, resulting in low network usage

- Effective data dispersion
- The effective use of remote sensing and control
- Prompt and effective message delivery
- Minimizes power consumption, which is beneficial for the linked devices, and maximizes network capacity.
- Data transmission is quick, efficient, and lightweight because MQTT messages have small code footprint. These control messages have a fixed header of size 2 bytes and payload message up to size 256 megabytes.

Introduction about Blynk IoT

Blynk is a software platform that allows users to connect their hardware to the cloud and manage connected devices remotely. Blynk is designed for the Internet of Things (IoT) and can be used for personal projects or commercial products.

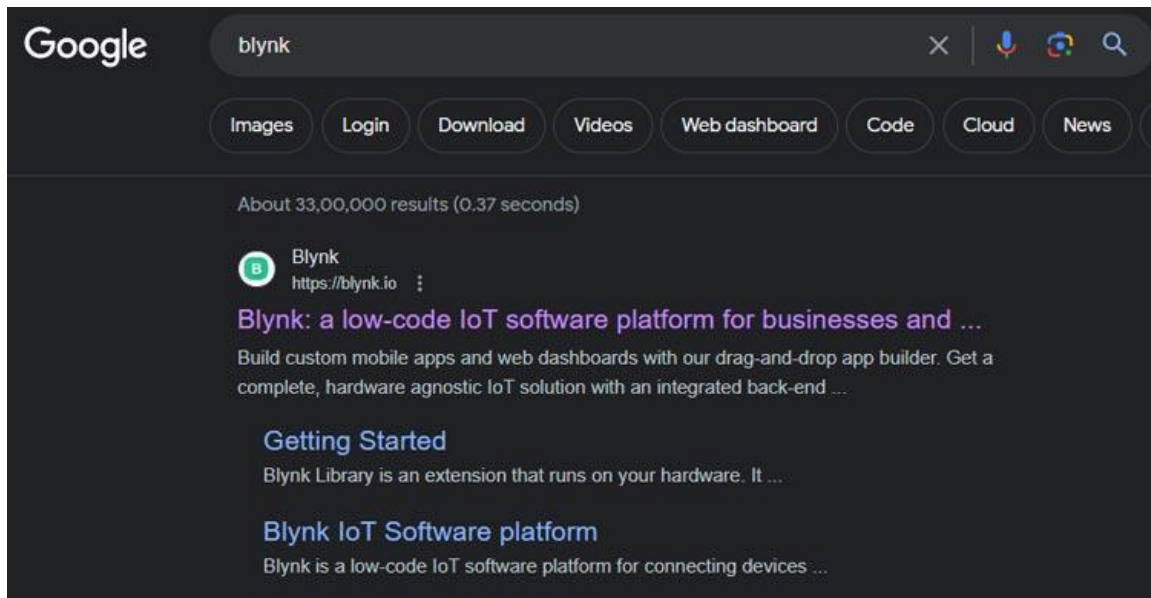
Here are some features of Blynk:

- No-code approach: Blynk allows users to build and scale IoT without coding. For example, users can control an LED or motor from their phone with just a few clicks.
- Remote management: Users can remotely control devices from anywhere, receive notifications, and analyze data.
- Multi-tenancy: Users can configure roles and permissions to control who has access to data.
- Scalability: Blynk can handle thousands of devices and is used by businesses of all sizes.
- Open-source: Blynk's server is open-source and can be run locally or on Blynk Cloud.
- Blynk App: Users can create interfaces for their projects using widgets.
- Blynk Libraries: Users can communicate with the server and process commands for popular hardware platforms.

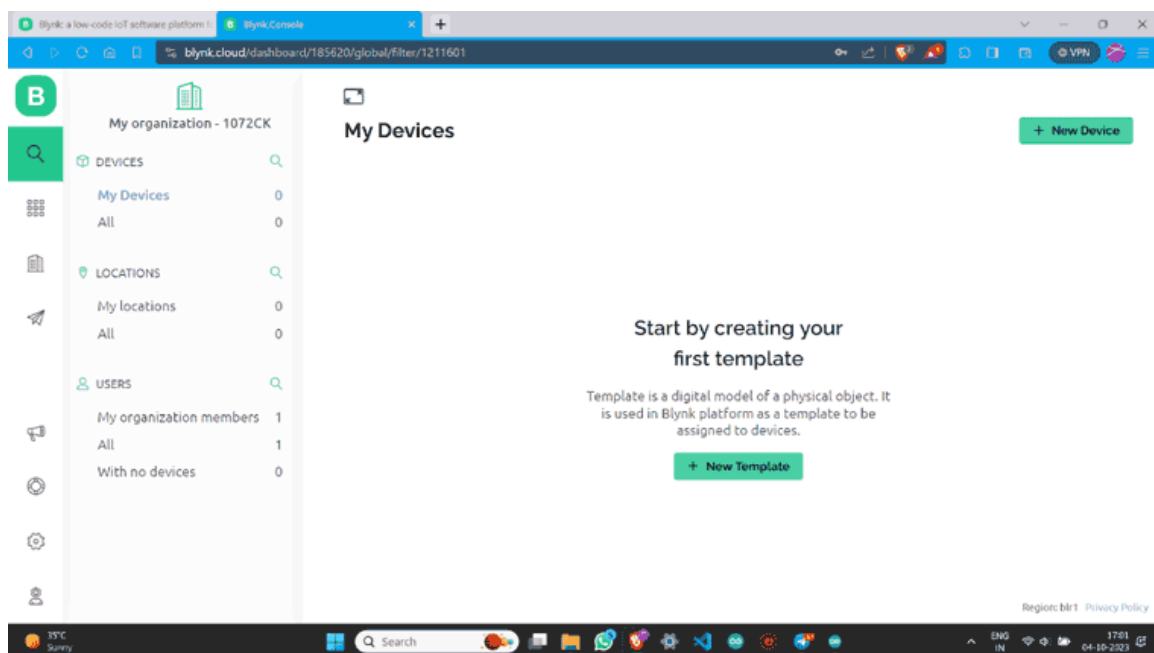
Blynk is free for personal use and prototyping. Businesses can purchase subscriptions to publish Blynk-powered apps for their products.

Setup Blynk 2.0 App

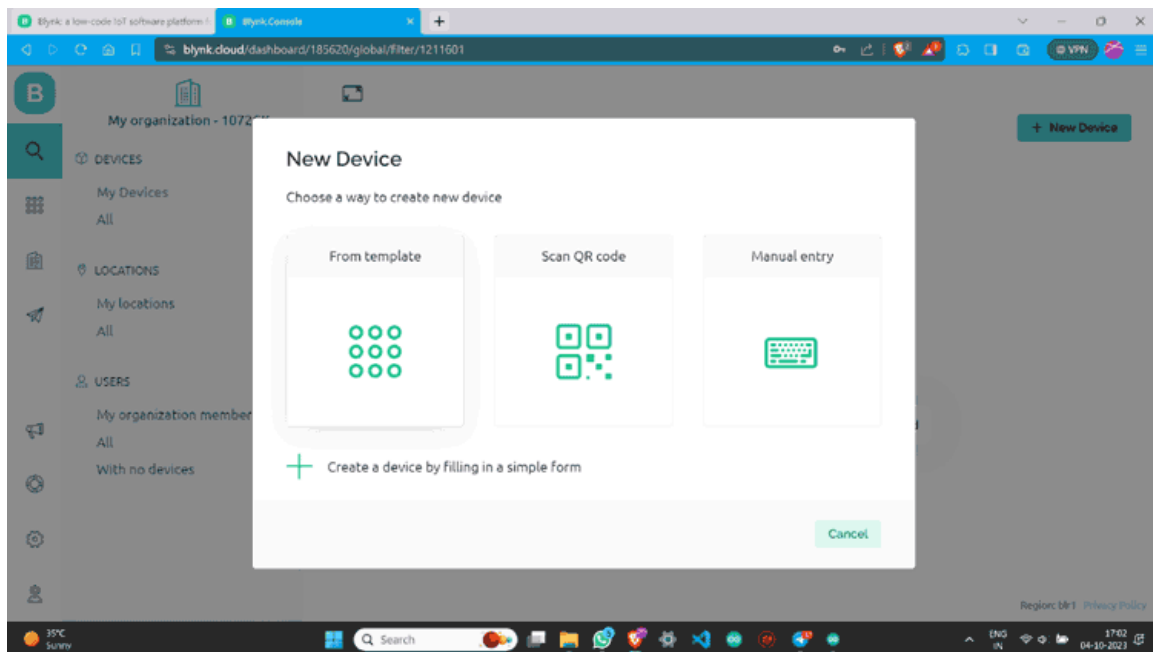
Step 1: Go to the Blynk website and click the "Sign Up" button. Enter your email address and create a password.



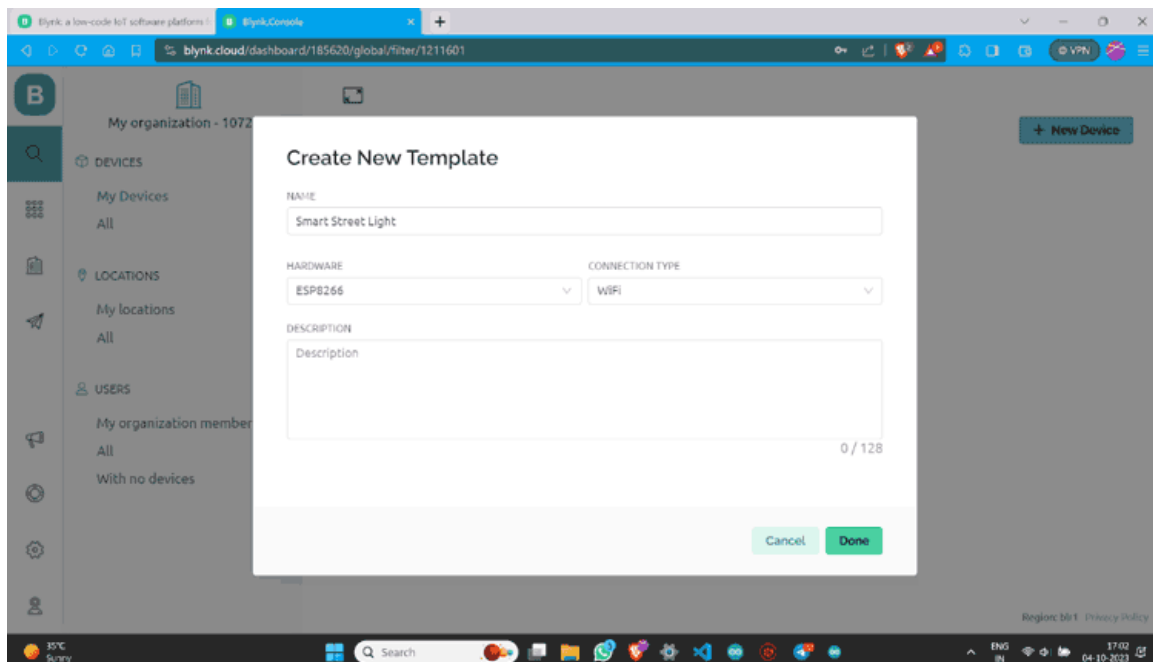
Step 2: Once you have signed up on the Blynk and logged in, tap the "+ New Template" button to create a new template.



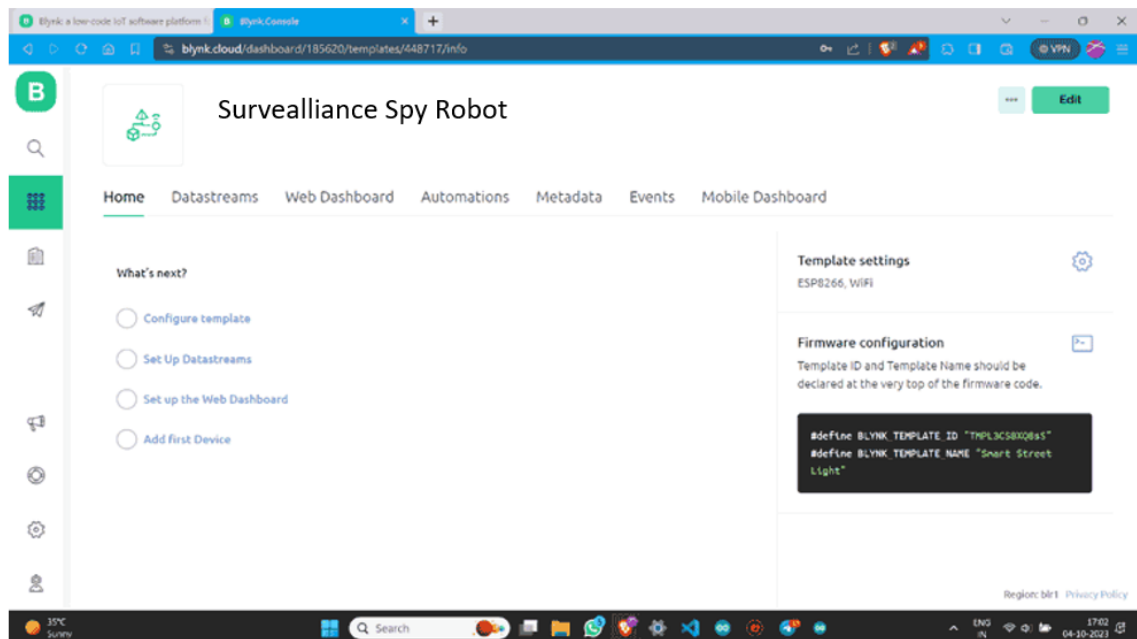
Step 3: Click “From template”, select the template, and create.



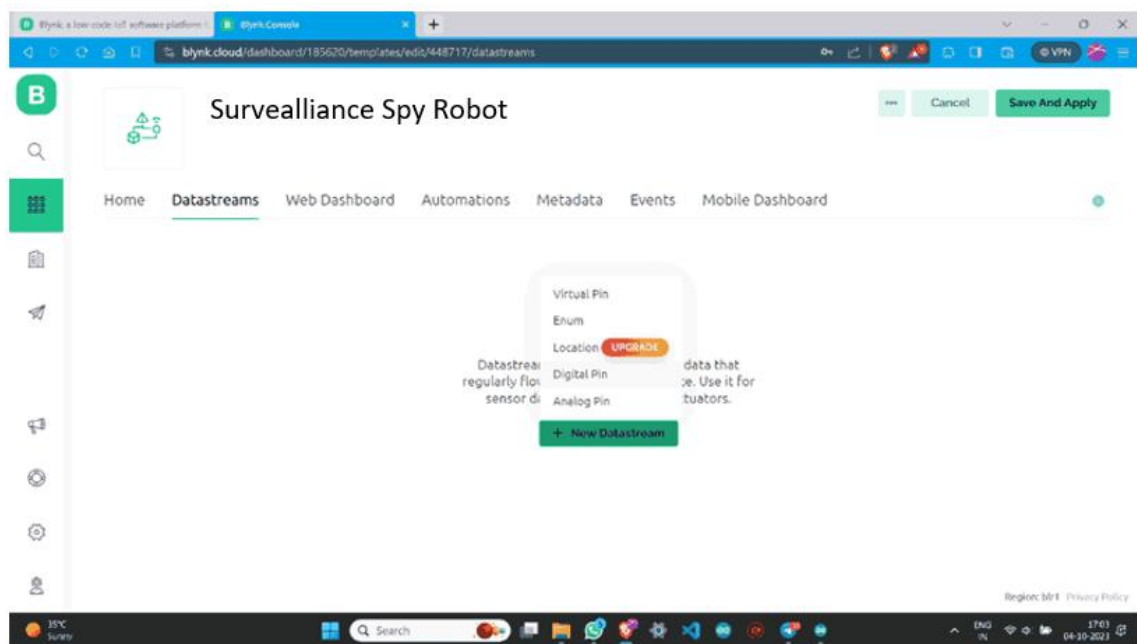
Step 4: Name the template, select the Development Board, set Wi-Fi as the connection type and then save this template.



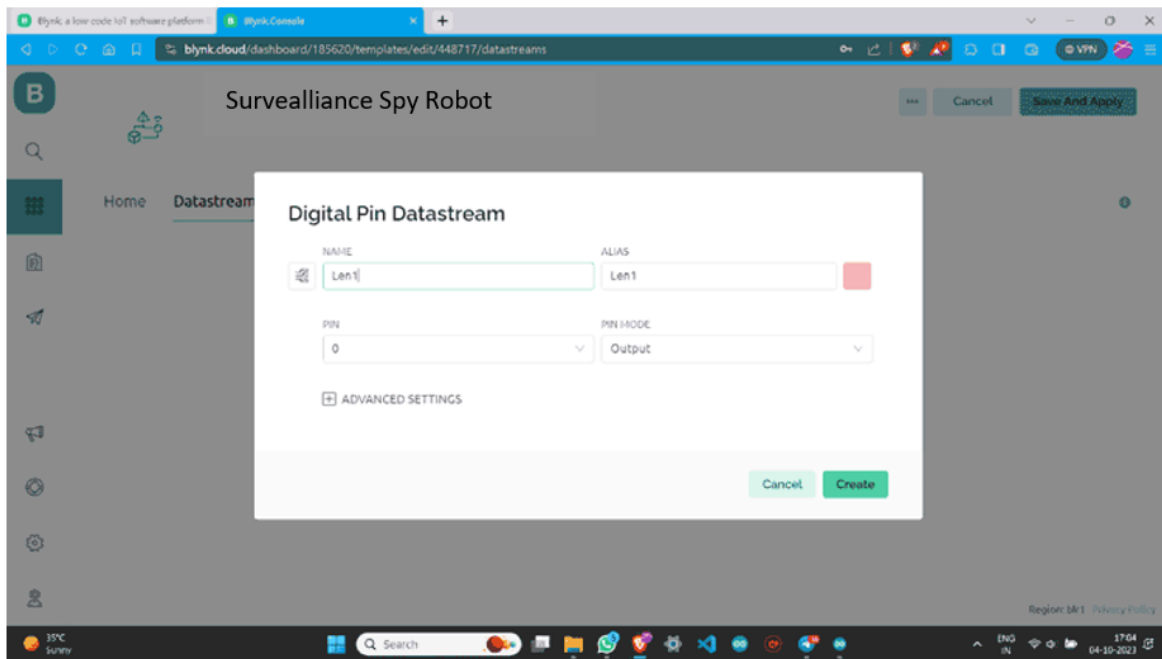
Step 5: Once a new template is created you will see the below screen on your Blynk account



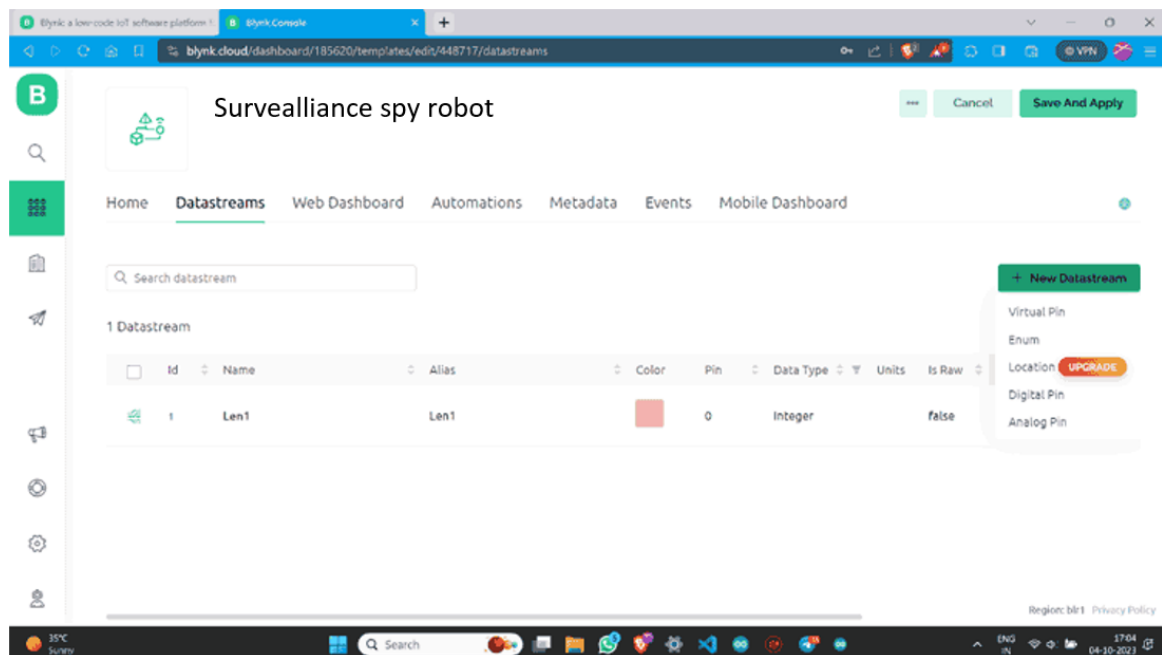
Step 6: Go to DataStream and click on "+ New DataStream". Then Choose "Digital Pin".



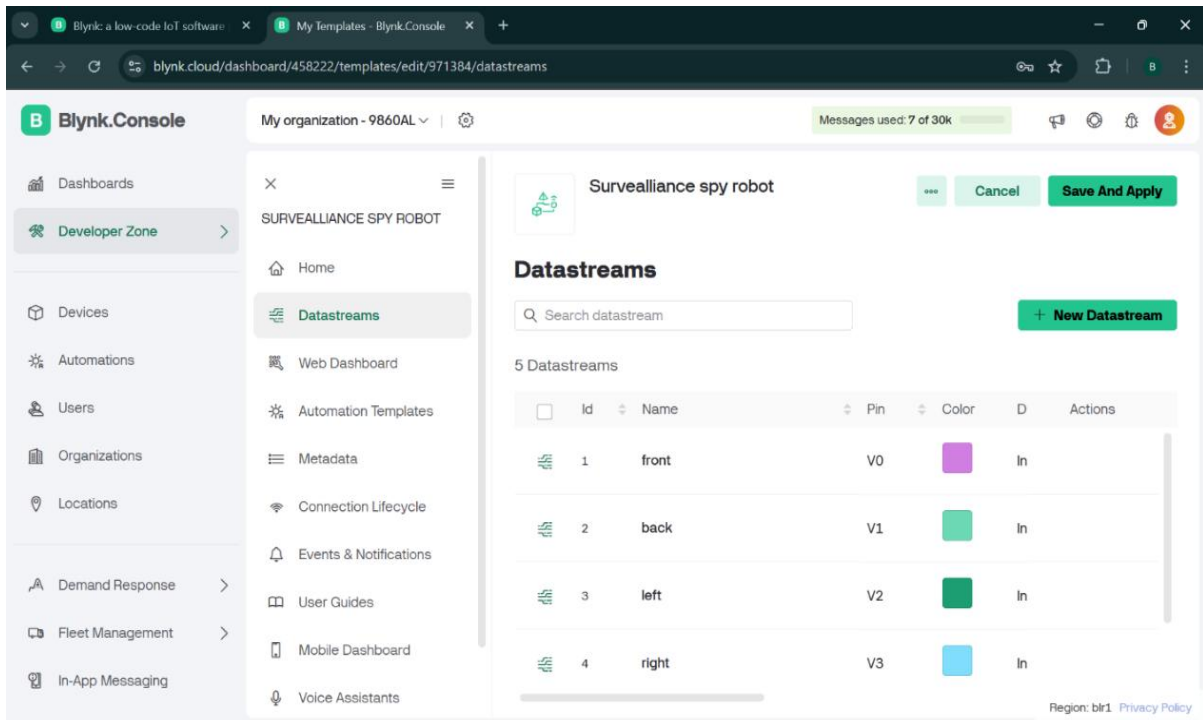
Step 7: Name DataStream and select pin. Assign its PIN MODE as Output and Create.



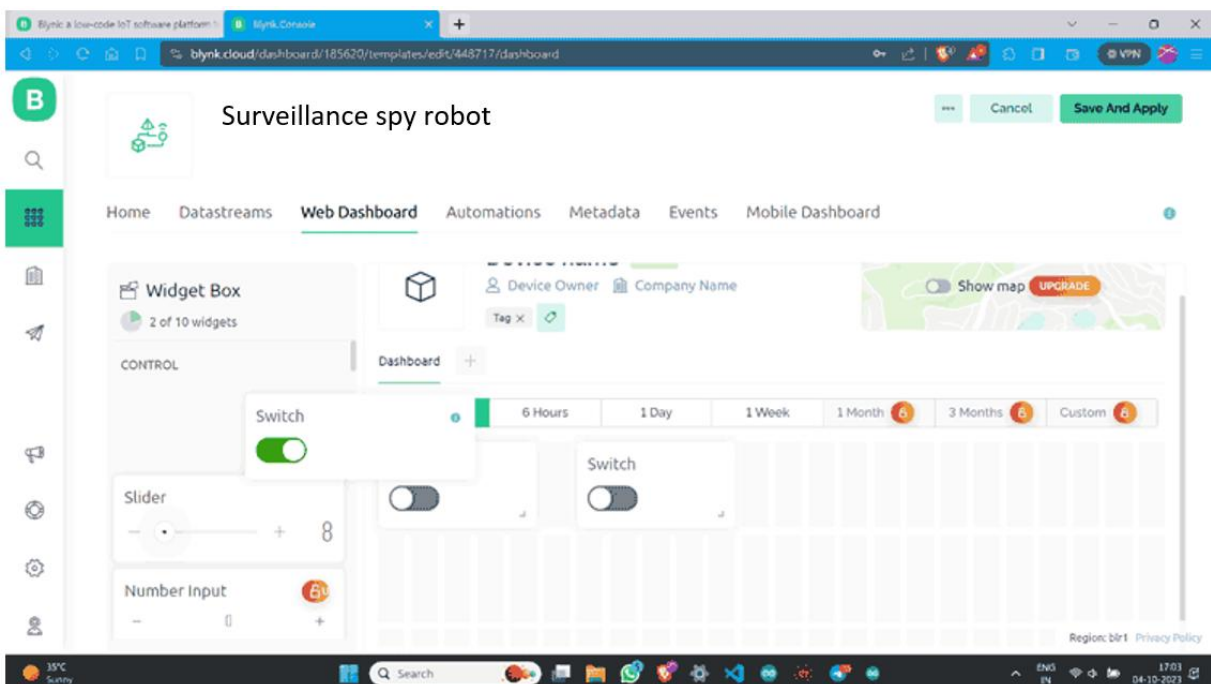
Step 8: Once a DataStream is created on the Blynk website, it will look something like this below



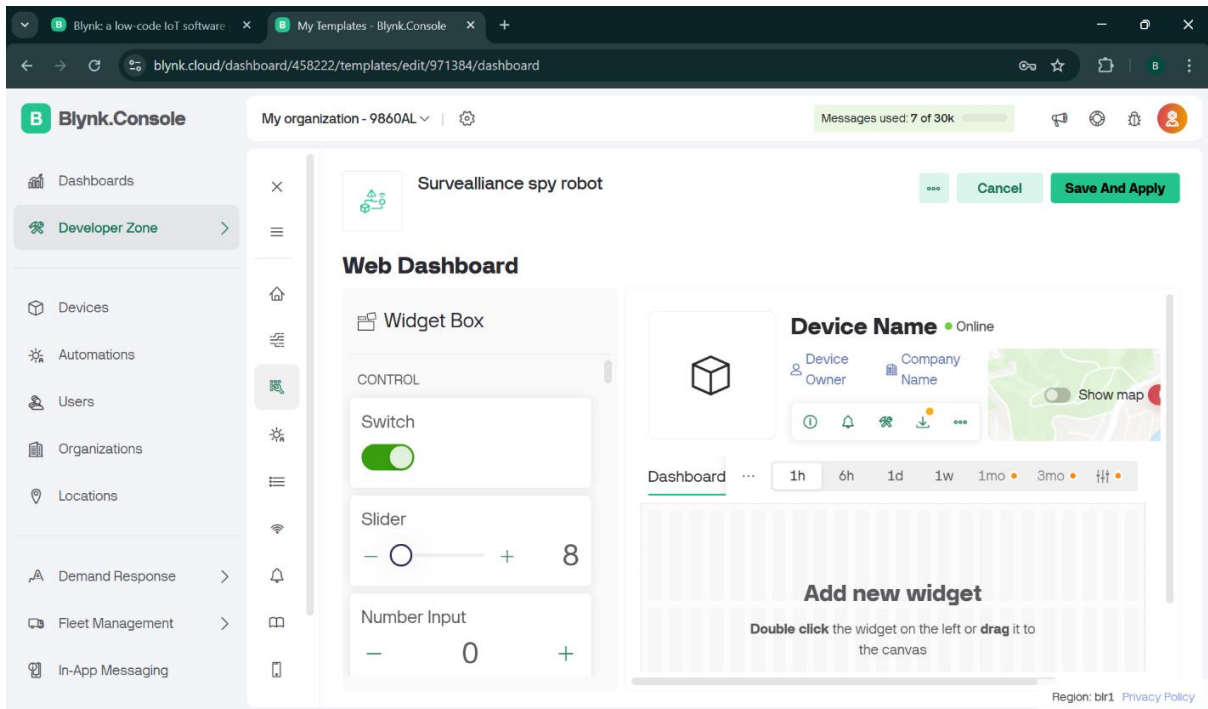
Step 9: I have made one more data stream with the same values.



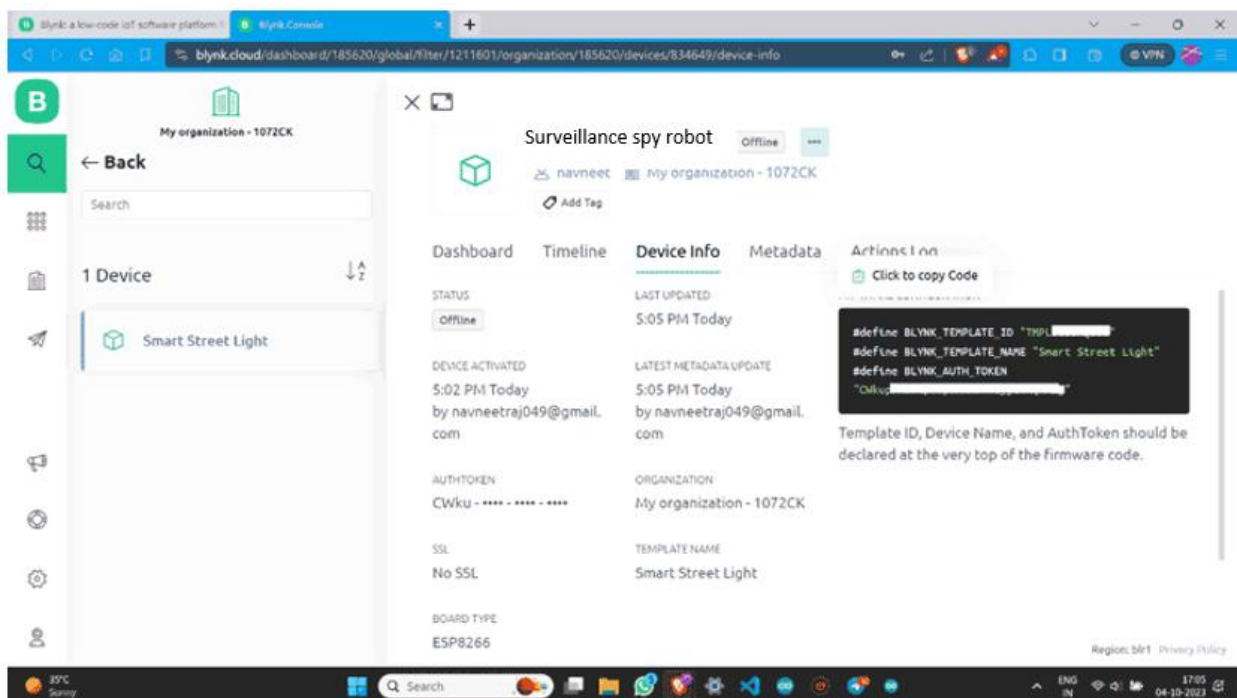
Step 10: Go to “Web Dashboard” and add Switch widgets.



Step 11: Click on the Gear button, give values according to your choice, and save.



Step 14: Now you can see the device. Click on “Device Info” and copy the Auth Token

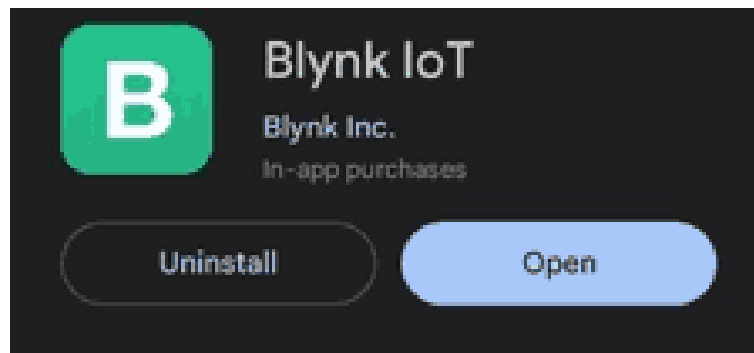


Blynk 2.0 Application Setup

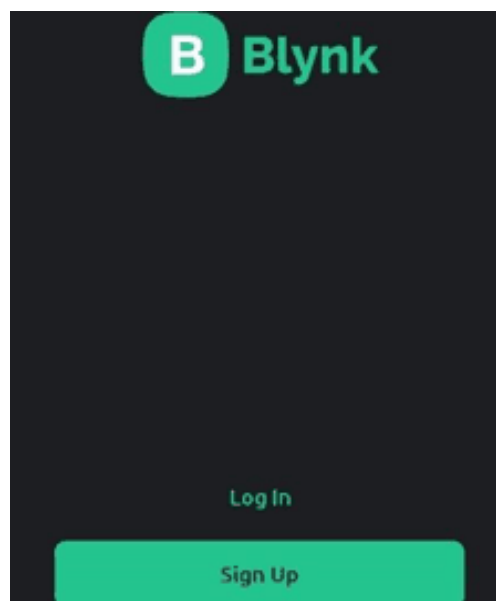
Now after Web setup, we have to make a Mobile App Interface so that we can able to control it from any Android or IOS

Step 1: Download the Blynk app.

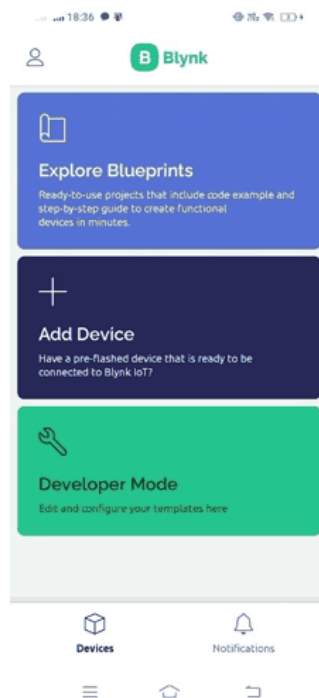
The Blynk app is available for both IOS and Android devices. You can download the app from the App Store or Google Play.



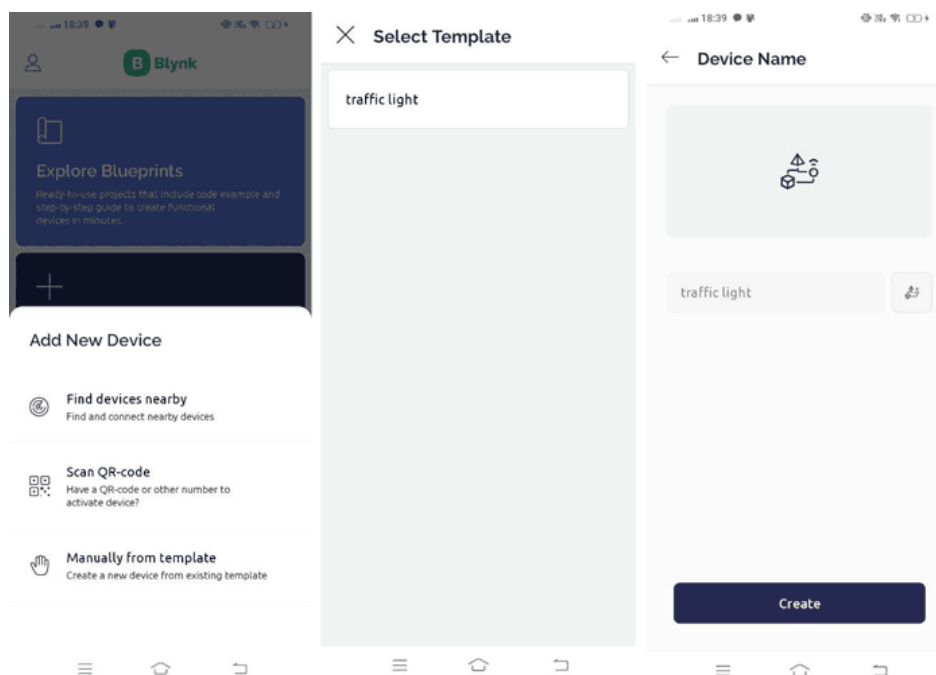
Step 2: Now open the Blynk IOT app and login with the credentials.



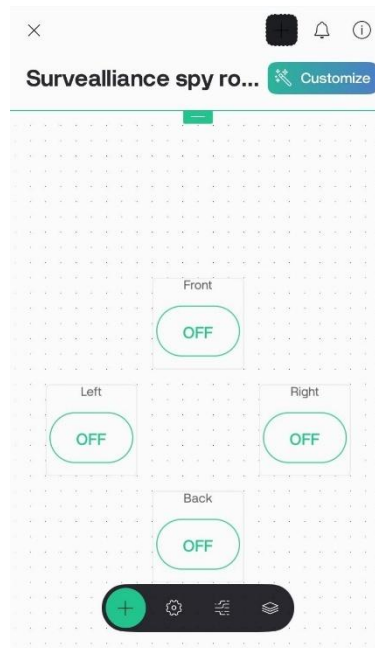
Step 3: After logging in click on “Add Device”.



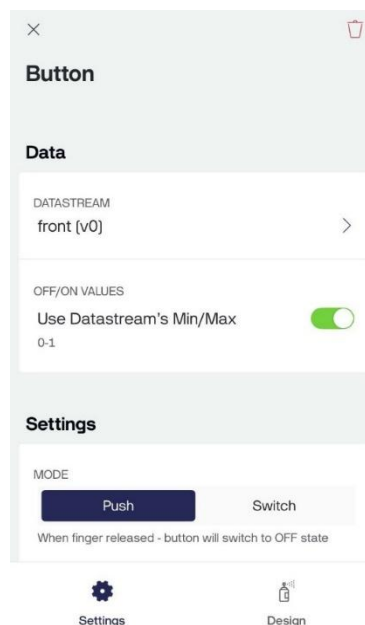
Step 4: Click “Manually from template”, select the device we already created, and then create.



Step 5: Now you are on the dashboard of the device then click on the tool button to enter in Developer Mode.

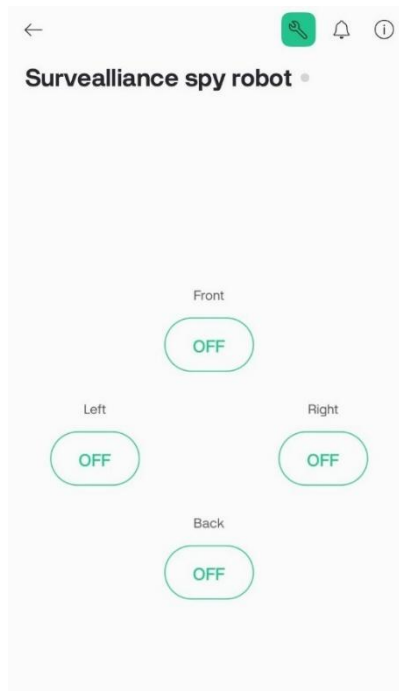


Step 6: Click on the button icons to assign values like DataStream and Mode. You can also change its design and looks.

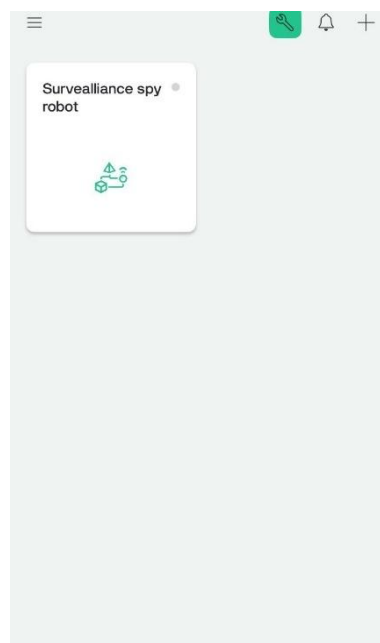


Step 7: You Have successfully created a device dashboard

.



Step 8: You can check the device status also.



Setup Arduino IDE to Node MCU ESP8266 board to use Blynk

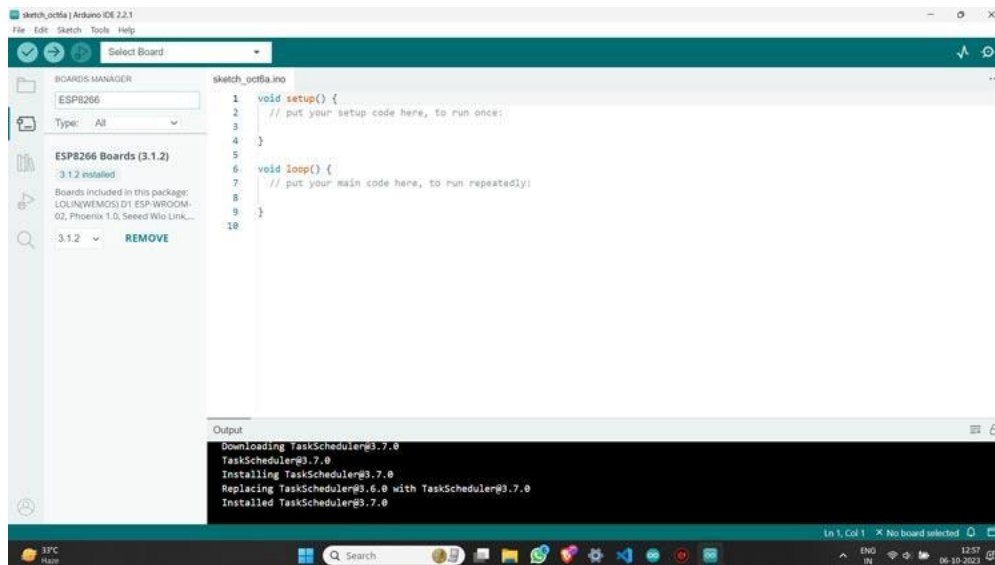
Now we have to set our Arduino IDE to program the Node MCU ESP8266 board.

Step 1: In Arduino IDE go to the “BOARDS MANAGER” and install ESP8266.

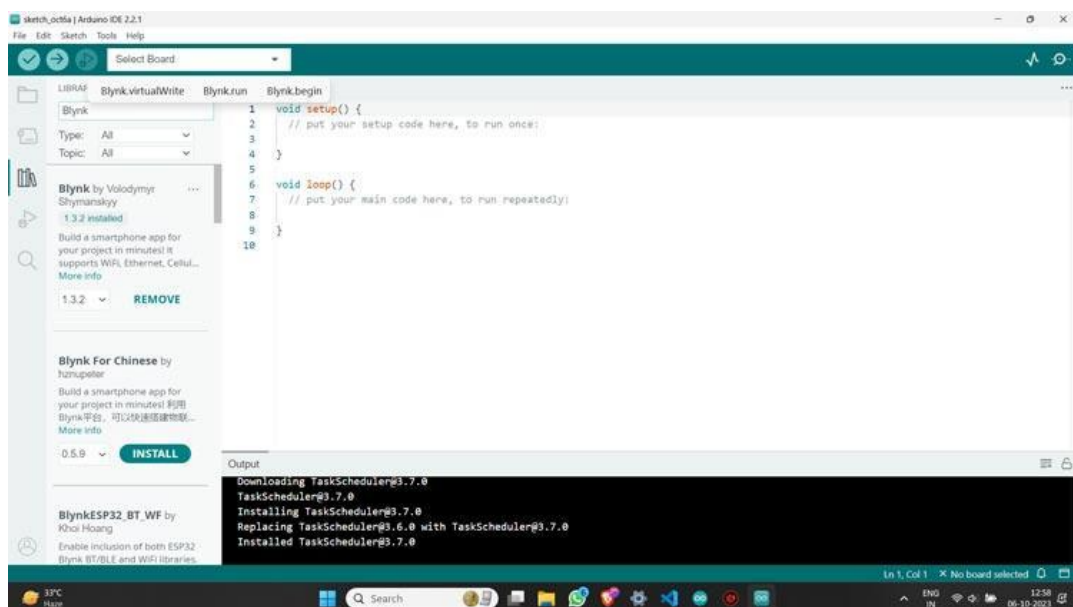
Install Blynk Library in Arduino IDE

to set our **Arduino IDE** to program the **Node MCU ESP8266 board**.

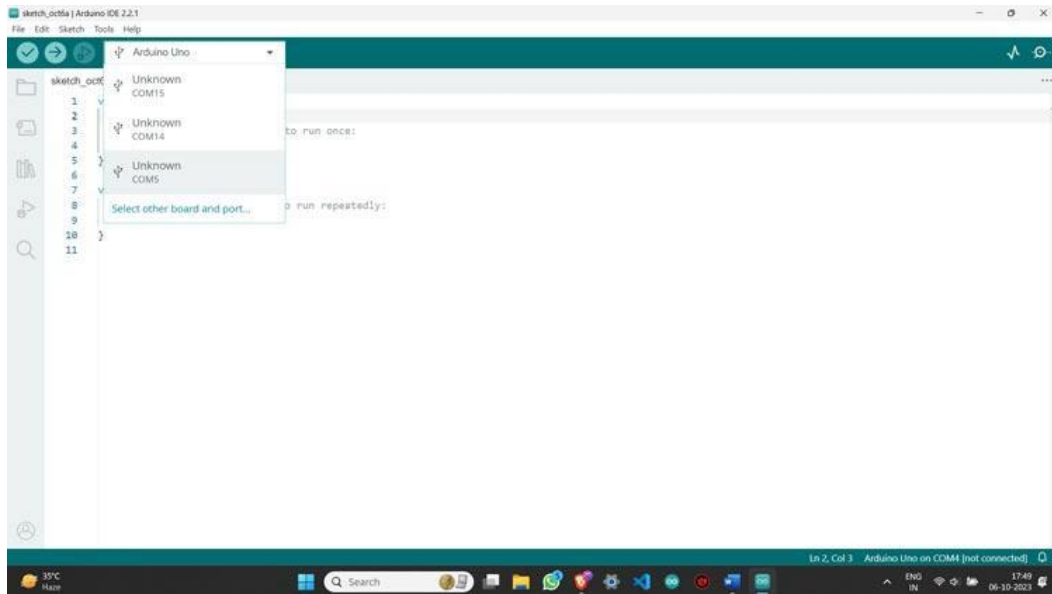
Step 1: In Arduino IDE go to the “**BOARDS MANAGER**” and install **ESP8266**.



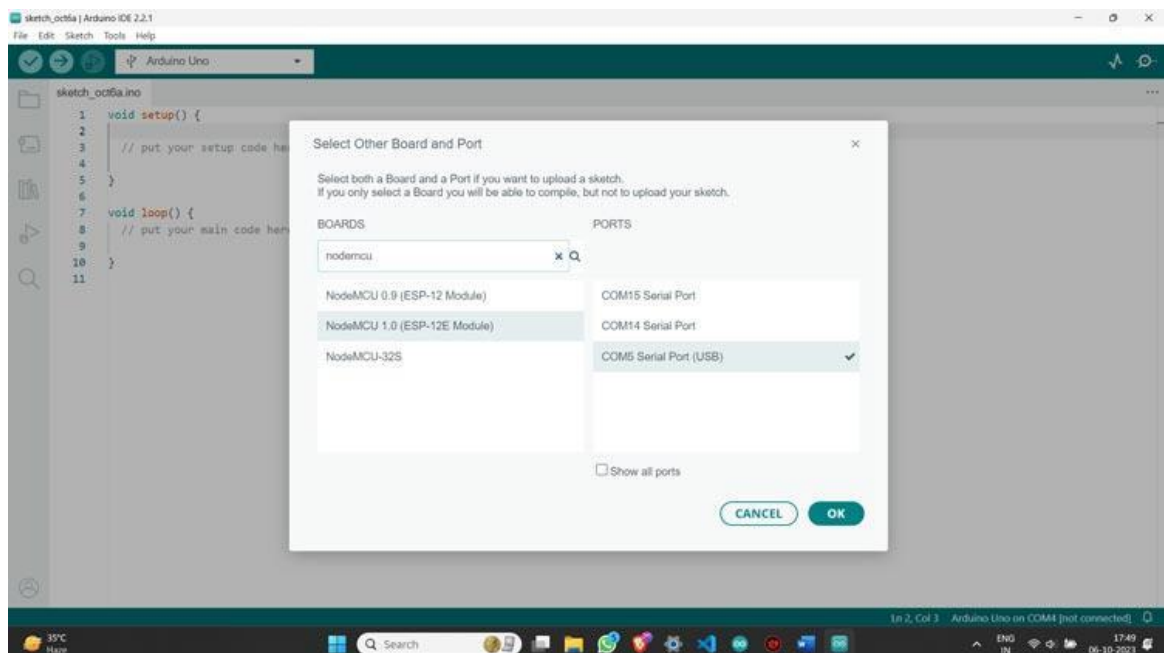
Step 2: Then go to the “**LIBRARY MANAGER**” and install the **Blynk** library.



Step 3: Plug your Node MCU ESP8266 board into the PC using a **USB cable** and then **select your board and port**.



Step 4: Your Arduino IDE is now **ready** to program your board for using Blynk app



CHAPTER 6

WORKING PRINCIPLE OF IDEATION

6.1 Project Working Processes:

The Surveillance Spy Robot operates based on a combination of embedded systems and wireless communication. The working principle follows these steps:

1. The ESP32 microcontroller receives movement commands from the IoT-based mobile application.
2. The motor driver processes these commands and controls the robot's movement accordingly.
3. The ESP32-CAM module captures real-time video and transmits it to the user's mobile device.
4. The LM2596 power supply module regulates the voltage to ensure efficient operation of the system components.
5. The user can remotely navigate the robot and monitor surroundings through the Android app.

CHAPTER 7

OUTPUT & RESULTS

7.1 Project Photos

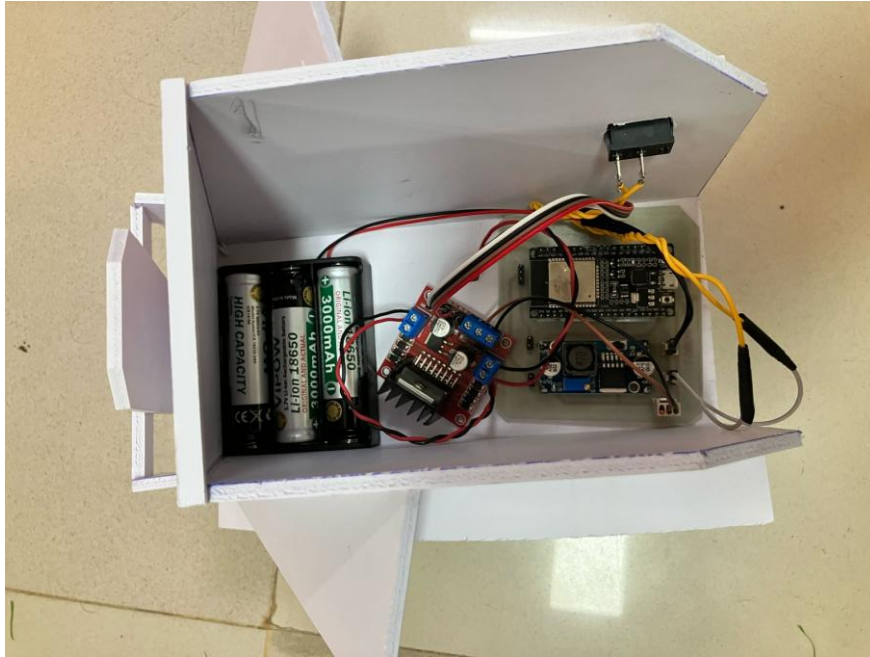


Fig: 7.1: Project photos



Fig: 7.2: Project Result

CHAPTER 8

CONCLUSION

8.1 Project Conclusion:

In this project, we successfully designed and developed a cost-effective and remotely controlled **Surveillance Spy Robot** using the **ESP32 microcontroller, ESP32-CAM module, motor driver, and LM2596 power supply module**. The robot is capable of real-time video surveillance and remote navigation via a **Wi-Fi-based Android application**, making it an ideal solution for security, military reconnaissance, industrial monitoring, and home surveillance applications.

The **prototype addresses the limitations of traditional surveillance systems** by providing mobility, flexibility, and real-time monitoring without the need for fixed infrastructure. Unlike conventional CCTV systems, which require extensive wiring and maintenance, our surveillance robot offers a portable and autonomous alternative that can be deployed in hazardous or inaccessible environments.

CHAPTER 9

FUTURE SCOPE

9.1 Future addons or Updates in project:

The future scope of the **Surveillance Spy Robot** is vast, with several enhancements that can improve its efficiency, intelligence, and usability. One of the primary improvements is integrating **GSM modules** to enable remote access over the internet, allowing users to control the robot and monitor live video feeds from anywhere. Additionally, incorporating **AI-based image processing** can enhance security by enabling **facial recognition, motion tracking, and anomaly detection**, making the robot more autonomous in identifying potential threats. Autonomous navigation is another crucial advancement, where the inclusion of **ultrasonic sensors, LiDAR, or GPS** can enable the robot to move independently while avoiding obstacles, making it suitable for large-scale surveillance operations.

To enhance power efficiency, **solar-powered charging systems** and **automated docking stations** can be introduced, ensuring uninterrupted surveillance operations. Expanding the project to a **multi-robot surveillance system** could further improve security by allowing multiple robots to communicate and work together for large-area monitoring. Moreover, advanced **security features such as encrypted communication and tamper detection mechanisms** can be implemented to prevent unauthorized access and interference. The robot can also be deployed for **disaster response, search and rescue missions, and environmental monitoring** in hazardous areas where human presence is risky.

Beyond security applications, this technology can be utilized in **industrial automation, smart city traffic monitoring, and border surveillance**, making it a versatile tool for both government and private sectors. In conclusion, by integrating **AI, IoT, autonomous navigation, and renewable energy solutions**, the Surveillance Spy Robot can evolve into a highly efficient and intelligent security system, revolutionizing modern surveillance technology.

CHAPTER 10

SOURCE CODE & RESOURCES

The IoT-Based Surveillance Spy Robot is a smart security system that enables remote monitoring using Blynk and ESP32. It allows real-time motion control via a mobile app, featuring live video streaming and obstacle detection for enhanced security. This project is ideal for home security, military surveillance, and industrial monitoring.

10.1 CODE

```
#define BLYNK_TEMPLATE_ID "TMPL3no8s_h2P"
#define BLYNK_TEMPLATE_NAME "Surveillance spy robot"
#define BLYNK_AUTH_TOKEN "xEWHJvqEOcOc_yrti3l3gC4_zqTYG4LE"

#define BLYNK_PRINT Serial

#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>

// Your WiFi credentials.
// Set password to "" for open networks.
char ssid[] = "robotics";
char pass[] = "makeindia";
int f, r, b, l, servo;
#define m1p 19
#define m1n 18
#define m2p 17
#define m2n 16

// This function will be called every time Slider Widget
```



```

// in Blynk app writes values to the Virtual Pin 1
BLYNK_WRITE(V0) {
    f = param.asInt(); // assigning incoming value from pin V1 to a variable
}
BLYNK_WRITE(V3) {
    r = param.asInt(); // assigning incoming value from pin V1 to a variable
}
BLYNK_WRITE(V1) {
    b = param.asInt(); // assigning incoming value from pin V1 to a variable
}
BLYNK_WRITE(V2) {
    l = param.asInt(); // assigning incoming value from pin V1 to a variable
}
BLYNK_WRITE(V5) {
    //spray = param.asInt(); // assigning incoming value from pin V1 to a variable
    //digitalWrite(pump,spray);
}
void setup() {
    // Debug console
    Serial.begin(115200);
    pinMode(m1p, OUTPUT);
    pinMode(m1n, OUTPUT);
    pinMode(m2p, OUTPUT);
    pinMode(m2n, OUTPUT);
    //pinMode(pump, OUTPUT);
    digitalWrite(m1p, 0);
    digitalWrite(m1n, 0);
    digitalWrite(m2p, 0);
    digitalWrite(m2n, 0);
    Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
}

```

```

}

void loop() {
  Blynk.run();
  if(f==1 && b==0 && r==0 && l==0)
  {
    Serial.println("front");
    digitalWrite(m1p, 1);
    digitalWrite(m1n, 0);
    digitalWrite(m2p, 1);
    digitalWrite(m2n, 0);
  }
  else if(f==0 && b==1 && r==0 && l==0)
  {
    Serial.println("back");
    digitalWrite(m1p, 0);
    digitalWrite(m1n, 1);
    digitalWrite(m2p, 0);
    digitalWrite(m2n, 1);
  }
  else if(f==0 && b==0 && r==1 && l==0)
  {
    Serial.println("right");
    digitalWrite(m1p, 0);
    digitalWrite(m1n, 0);
    digitalWrite(m2p, 1);
    digitalWrite(m2n, 0);
  }
  else if(f==0 && b==0 && r==0 && l==1)
  {
    Serial.println("left");

```

```

digitalWrite(m1p, 1);
digitalWrite(m1n, 0);
digitalWrite(m2p, 0);
digitalWrite(m2n, 0);
}
else
{
  Serial.println("stop");
  digitalWrite(m1p, 0);
  digitalWrite(m1n, 0);
  digitalWrite(m2p, 0);
  digitalWrite(m2n, 0);
}
}

```

10.2 Resources:

1. Libraries:

- **Servo.h** – Controls the **servo motor** for sensor rotation.
- **WIFI** – Handles **Wi-Fi communication** for ESP32.
- **BlynkSimpleEsp32.h** – Enables **Blynk IoT platform** integration.
- **ESP32Servo.h** – Alternative library for **controlling servos** with ESP32.
- **Arduino Json.h** – Useful for **data exchange** in JSON format with the Blynk app.

2. Hardware Components:

- **ESP32-WROOM-32** – Main **microcontroller** for processing & connectivity.
- **ESP32-CAM** – Captures **live video stream** & transmits it wirelessly.
- **L298N Motor Driver** – Controls **DC motors** for movement.
- **DC Motors** – Used for **forward, backward, left, and right motion**.
- **LM2596 Power Supply Module** – Provides a **stable voltage supply**.
- **Servo Motor (SG90 or MG995)** – Rotates **ultrasonic sensor** for scanning.

- **Power Supply (Battery or Adapter)** – 5V - 12V power source for ESP32 & motors.

3. **Blynk Platform:**

- **Blynk Virtual Pins** – Used for **remote control** of robot movements.
- **V0 - Forward, V1 - Backward, V2 - Left, V3 - Right**
- **Blynk Dashboard** – Displays camera feed, and control buttons.

4. **Tools:**

- **Arduino IDE** – To write, compile, and upload code to ESP32.
- **Blynk App** – For remote control and real-time data visualization.
- **Fritzing / Tinker cad** – For circuit diagram design & simulation.
- **ESP32-CAM Web Server Library** – If using ESP32-CAM for live streaming.

CHAPTER- 11

REFERENCES

1. R. P. and S. Bagwari, "IoT Based Military Assistance and Surveillance," 2018 International Conference on Intelligent Circuits and Systems (ICICS), pp. 340–344, 2018.
2. C. Zhang and J. Kim, "Video Object Detection With Two-Path Convolutional LSTM Pyramid," IEEE Access, vol. 8, pp. 151681–151691, 2020.
3. R. Harshitha, M. Hameem, and S. Hussain, "Surveillance Robot Using Raspberry Pi and IoT," IEEE International Conference on Design Innovations for 3Cs Compute Communicate Control (ICDI3C).
4. N. Patil, S. Ambatkar, and S. Kakde, "IoT based smart surveillance security system using raspberry Pi," 2017 International Conference on Communication and Signal Processing (ICCSP), pp. 344–348, 2017.
5. V. P. Solanki and S. Deshmukh, "Wi-Fi Based Home Surveillance Bot," 2019 5th International Conference on Computing Communication Control and Automation (ICCUBE), pp. 1–5, 2019.
6. M. F. Alvi, F. Junejo, M. W. Baloch, and A. J. Rajput, "Robocop; A Robust Surveillance Robot," IEEE International Conference on Advanced Research in Engineering Sciences (ARES).
7. B. N. Rao, R. Sudheer, M. A. Sadhanala, V. Tibirisetti, and S. Muggulla, "Movable Surveillance Camera using IoT and Raspberry Pi," 2020 11th International Conference on Computing Communication and Networking Technologies (ICCCNT), pp. 1–6, 2020.
8. A. U. Bokade and V. R. Ratnaparkhe, "Video surveillance robot control using a smartphone and Raspberry Pi," IEEE International Conference on Communication and Signal Processing (ICCSP).
9. P. Wei, L. Xianli, A. Jianyong, and W. Ping, "The Application Research on Military Internet of Things," 2020 17th International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP), pp. 187–191, 2020.